

BRIEFINGS

black hat

Invisible Threads: Remote building surveillance through encrypted Thread traffic analysis

Speakers: Béla Genge, Anca Delia Burduv

Contributor: Ioan Pădurean



OUR PREVIOUS RESEARCH ON MATTER IOT



- Brand new standard, adopted by **600+ major industry players** (Google, Apple, Samsung, Amazon, etc.)
- We found several design flaws and vulnerabilities in the Matter IoT standard
- Enabled effective DoS, device identification
- Exploit **required Wi-Fi access**



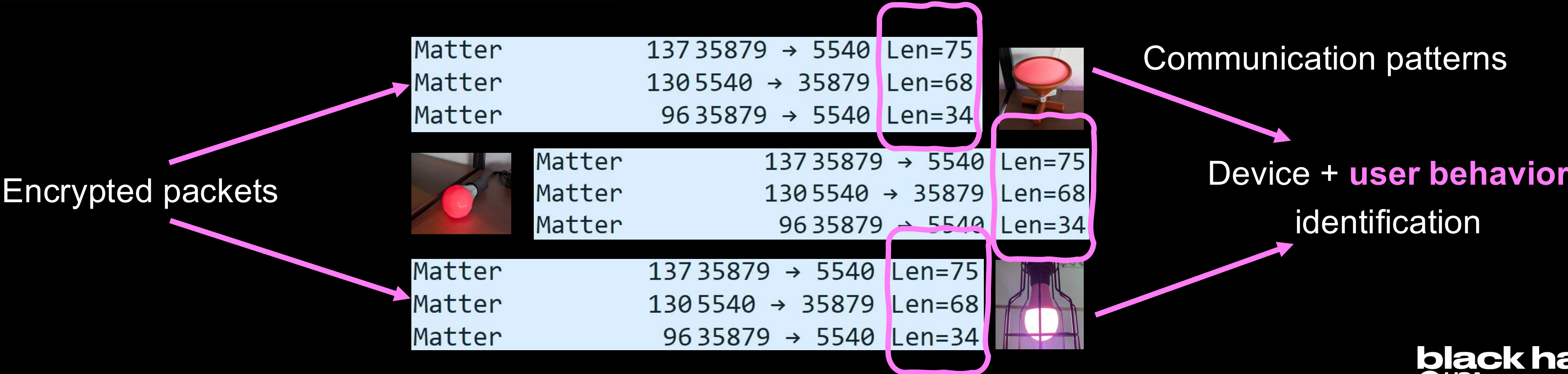
No replay protection



OUR LATEST BLACK HAT TALK: INFO LEAKS



- We accidentally found that Matter's encrypted traffic **leaks information** about devices
- Attackers can **accurately profile devices** on the network, conduct spying and profiling activities, **user behavior identification**, which can lead to other malicious activities
- Exploit **required Wi-Fi access**



NATURAL CURIOSITY: THIS RESEARCH

Application layer



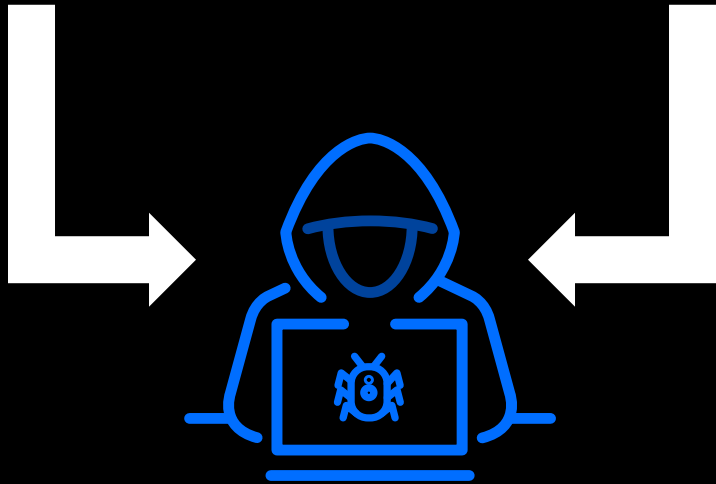
Communication / **transport** layer

34, 73, 42, 34, 59, 67, 34, 73, 42, 34, 59, 67

← Application layer patterns

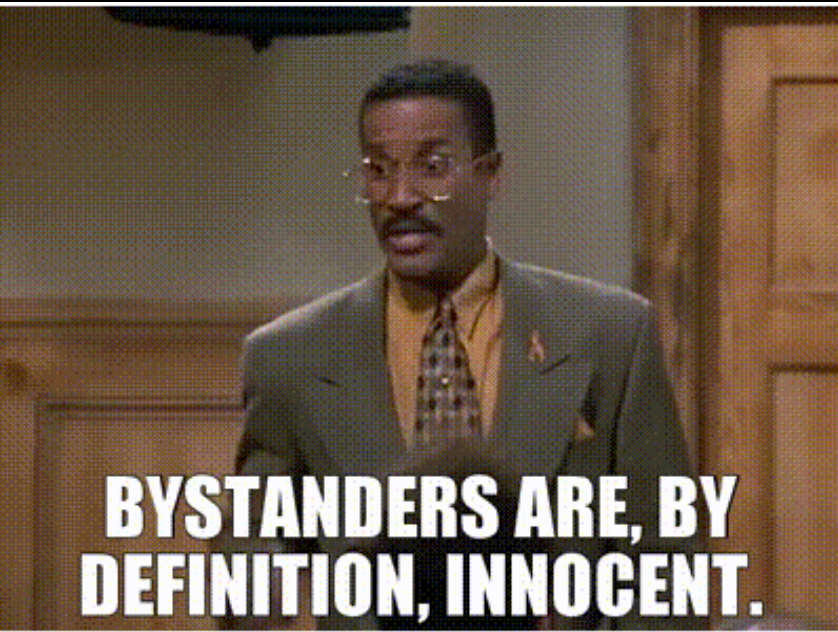


← **Transport** layer cryptographic protection

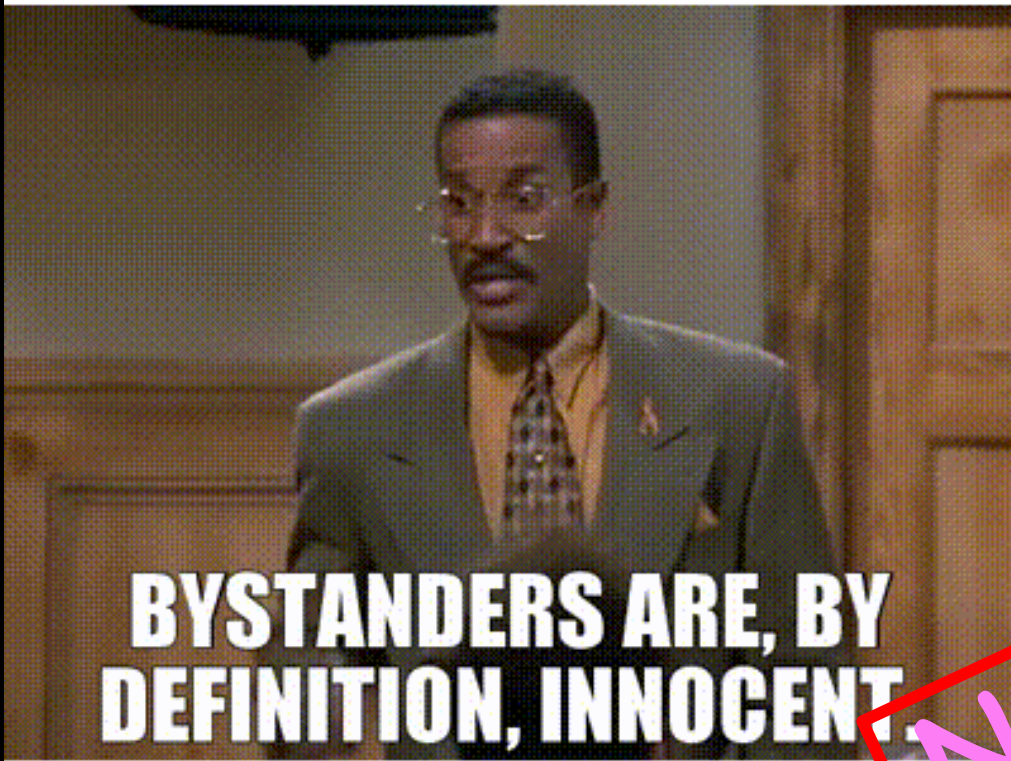


Is there protection to **prevent** further information leakage and exploitation?

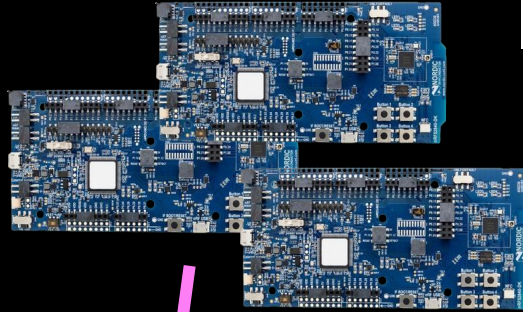
INNOCENT BYSTANDERS



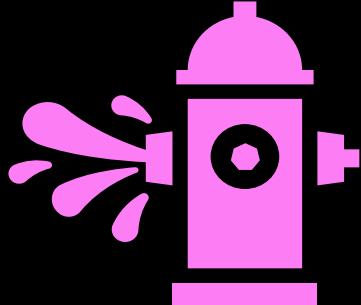
POSSIBLE EXPLOITATION



NOT



Information leakage?





Béla GENGÉ

Senior Security Researcher
Bitdefender

Leading research on Matter IoT security since 2023. He combines industrial innovation with academic exploration, diving deep into reverse-engineering proprietary protocols and making sense of the chaos in computer and IoT network traffic.



Anca Delia BURDUV

Junior Security Researcher
Bitdefender

She has a Bachelor's degree in Computer Science. Before joining Bitdefender, she interned on Arm's GNU Compiler Collection team, where she worked on Arm intrinsics.



Ioan PĂDUREAN

Security Researcher
Bitdefender

Research on Matter IoT since early 2024. He has a master's degree in Artificial Intelligence and is currently pursuing a PhD in IoT security.

Bitdefender®

OUTLINE

Buildings, motivation

1

Demo

2 Threads, information leaks

2

3

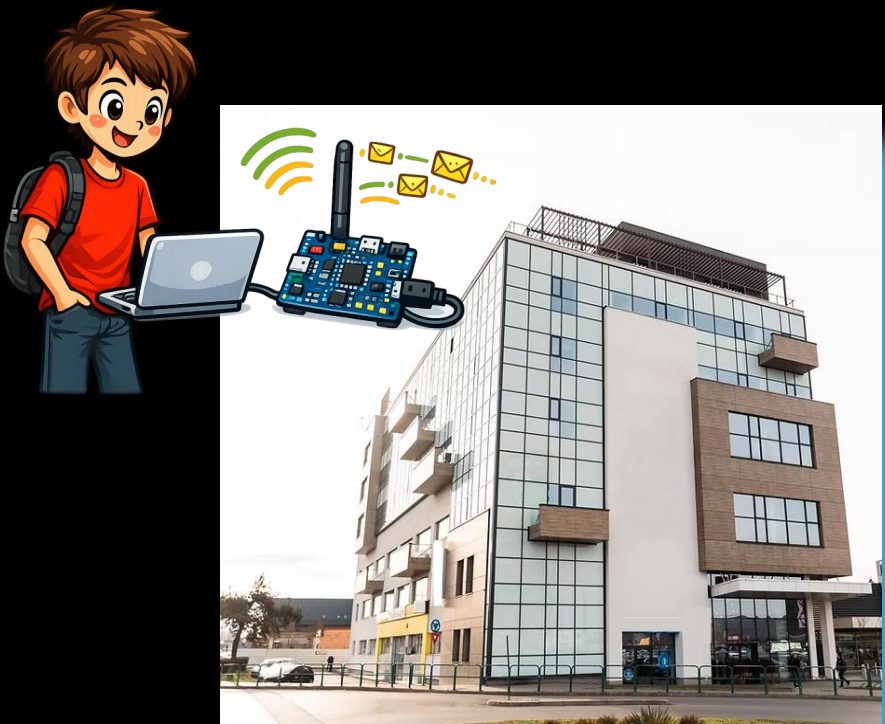
Infrastructure discovery

Demo

Demo

4

Exploitability



BUILDINGS

“ Buildings serve several societal needs – occupancy, primarily as shelter from weather, **security**, living space, **privacy**, to store belongings, and to comfortably live and work.



BUILDINGS – THE PROBLEM



The walls haven't changed. The threats have.

WHAT MOTIVATED THIS RESEARCH?

Notice anything interesting?



WHAT TRIGGERED THE RESEARCH

If we take a closer look



It just happens we had prior experience with such devices



WHAT IS THE TECHNOLOGY BEHIND IT?



Aqara U200 smart lock EU black with keypad

Easy installation over existing lock without drilling or modification and wide compatibility
Silent unlock mode, automatic alerts, automatic locking, temporary passwords and hands-free unlocking
Seamless integration with major smart home systems, such as Apple Home and Google Home

THREAD

matter

DescriptionFeaturesVideos (1)

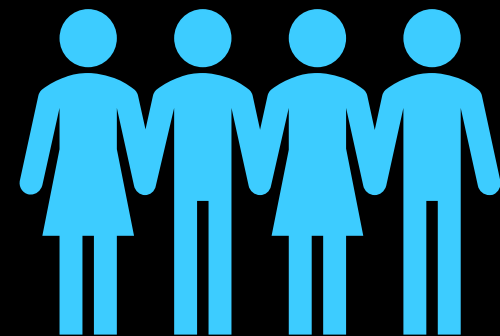
Upgrade your home to a keyless lifestyle with **the Aqara Smart Lock U200 Kit** , which easily installs over your existing EURO cylinder lock, without the need for drilling. This smart lock supports various access methods such as fingerprint, **NFC** , passcode and more, and integrates with **Apple Home** keys . Compatible with **Matter** , it is managed through the Aqara Home app and is built to last, with a six-month battery life and an IPX5 water resistance rating, making it ideal for both indoor and outdoor use.

- **Apple Home Keys and Multiple Unlock Methods** : The U200 offers a variety of convenient unlock methods, including Apple Home Keys (requires Apple 2-in-1 Matter controller and Border Router), fingerprint, passcode, NFC, and more. You can control the lock using the app via either built-in Bluetooth or Thread for remote access. Additionally, it supports temporary passwords, such as one-time and recurring passwords, for easy sharing with others. The device also allows the use of mechanical keys for emergencies.
- **Advanced Software Control** : U200 enhances home access with Silent Unlock Mode, automatic alerts and hands-free unlocking, ensuring security and peace of mind with features such as Activity Log, auto-lock and PIN Code Anti-spy Protection. It offers flexibility with Passage Mode, temporary passwords including offline passwords (One-Time Passwords) and periodic passwords available in the Aqara Home app, along with Do Not Disturb Mode.
- **Native Support for Matter over Thread** : The U200's Matter over Thread support ensures seamless integration with leading smart home systems, such as Apple Home and Google Home, improving user convenience by extending battery life and efficient remote management.
- **Rechargeable Battery and Multiple Power Options** : The U200's advanced engineering includes IPX5 waterproofing, an operating range of -15°C to 66°C, and multiple power sources: rechargeable Li-Ion for the lock (up to 6 months), AAA batteries for the keypad, and wired power capability. This ensures durable and reliable performance.

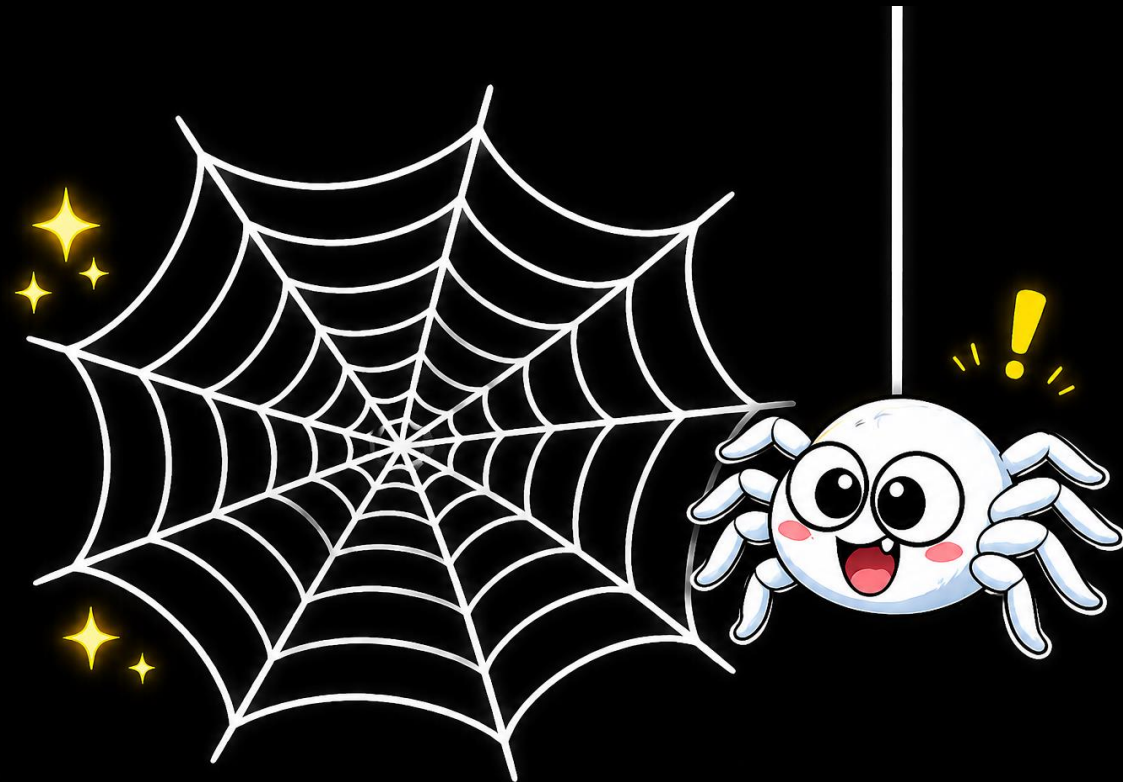
QUESTIONS OF INTEREST

If devices such as the smart door lock are not located on the street, is traffic still accessible from the street?

If so, what can "the (innocent) bystander" infer when analyzing such data leaks?

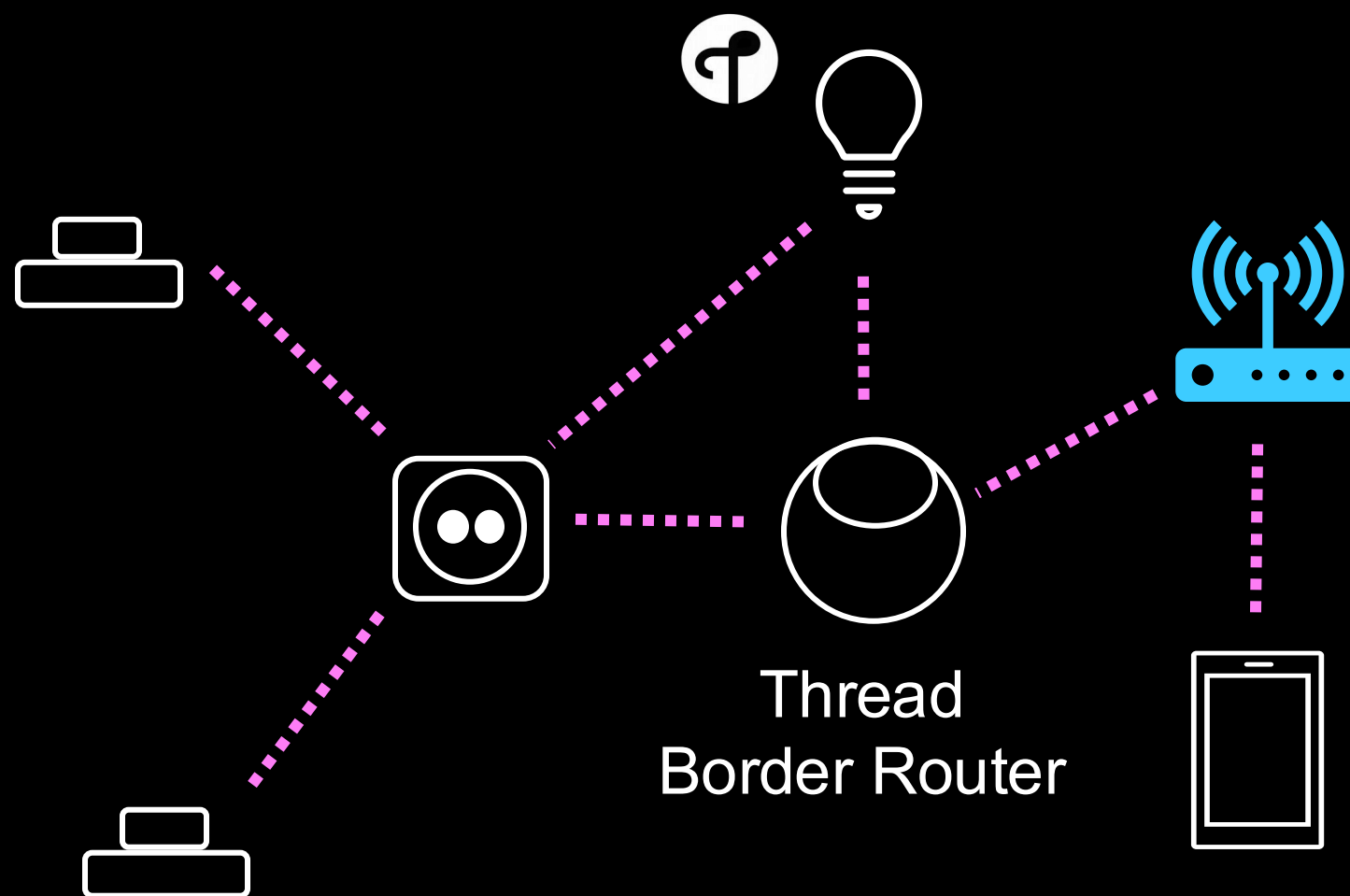


THREAD(S)



THREAD, THREAD & MATTER

- Thread:
 - Energy-efficient communication-layer for low-power devices
 - Self-healing mesh network
 - IPv6-based protocol for low-power, mesh networks
- Matter: the application-layer protocol



THREAD

HTTP, CoAP, MQTT, ...

DTLS

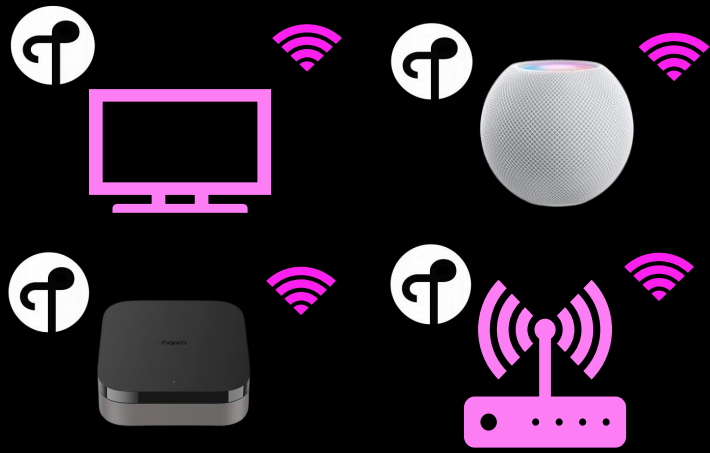
UDP

Distance Vector Routing

6LoWPAN (IPv6)

IEEE 802.15.4

EXAMPLE THREAD DEVICES



Border Routers:

Connect a Thread mesh network to external IP networks enabling communication between Thread devices and the outside world.



Router Eligible End Devices (REEDs):

Can be promoted to routers if the network needs more routing capacity.

(usually mains-powered)



Minimal End Devices (MEDs):

- Connect to a parent router but do not maintain full routing information, cannot become routers.

Sleepy End Devices (SEDs):

- Battery-powered devices that spend most of their time asleep to conserve energy.
- Rely on a parent router for message buffering cannot become routers.

WHAT IS THE BIG DEAL ABOUT THREAD?

Thread **progressed a lot** from its first release, it is not only viewed as a standard for smart homes

It is moving towards **commercial smart buildings**

Energy and industrial applications are on the horizon



16. Sep 2024

Thread Becomes Partner of the KNX-IoT Startup Incubator Program

Source: <https://www.knx.org/knx-en/newsroom/news/press/20240916-thread-becomes-partner-of-the-knx-iot-startup-incubator-program/>



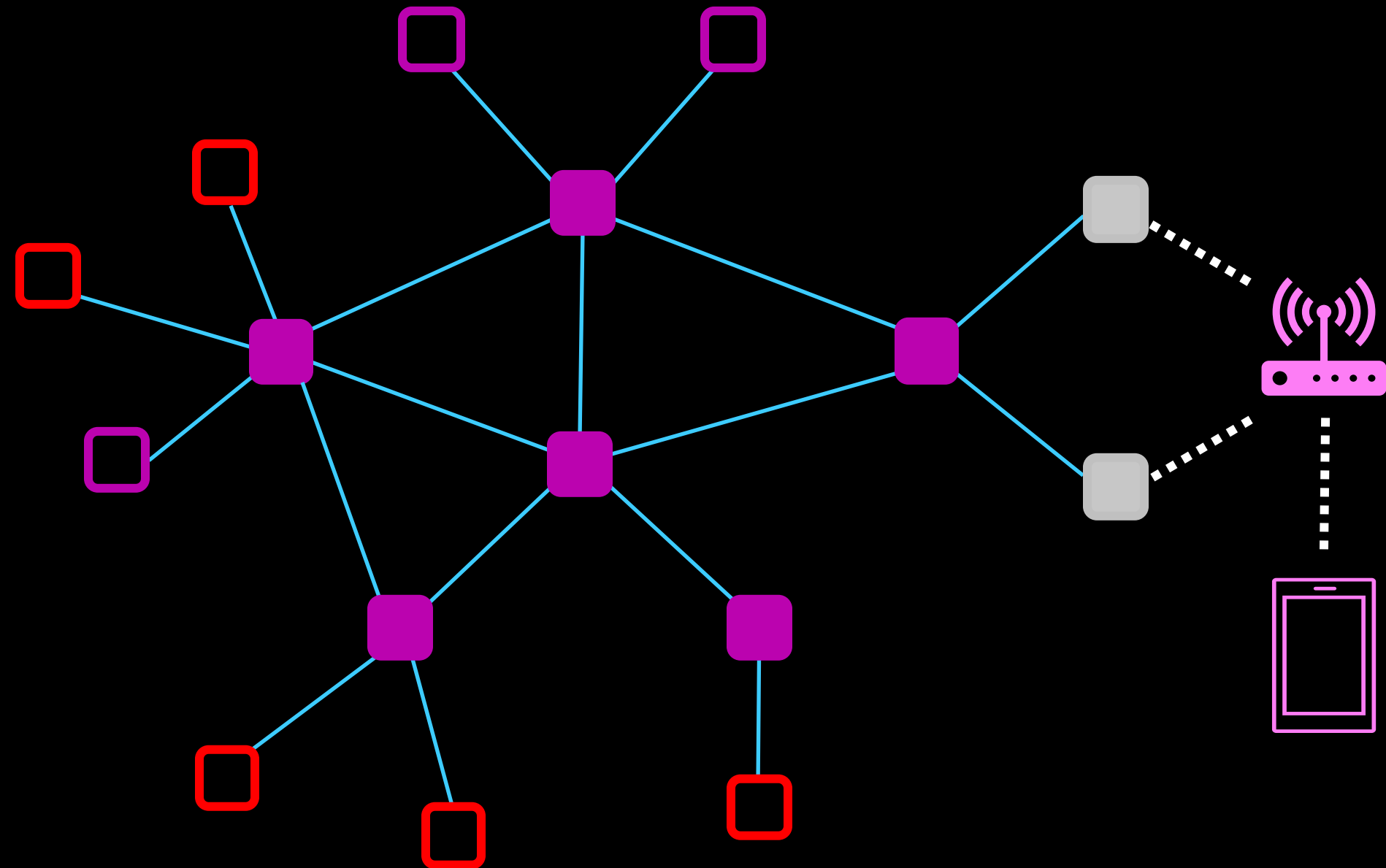
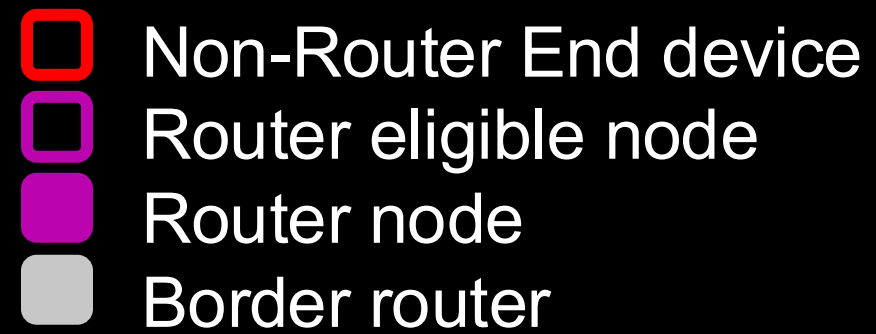
PRESS RELEASES

Thread Group Certifies IoT Hub for Smart Commercial Buildings and New Components

The WideSky Hub is the first certified and widely available Built on Thread product for commercial buildings, energy and industry applications. WideSky's wireless hub uses Thread to cost-effectively and reliably access valuable data across subsystems, devices and sensors without cables. Thread also ensures scalability and compatibility for the data infrastructure monitoring and controlling solutions of the future.

SELF-HEALING ARCHITECTURE

- Scalability of up to 10K Thread devices
- Self-healing automates reconfiguration



CHANNELS AND SIGNAL RANGE

- Thread uses the IEEE 802.15.4 2.4 GHz band from approximately 2405 MHz to 2480 MHz
- Possible channels:
 - 11: 2405 MHz
 - ...
 - 25: 2475 MHz
- Signal can range from 15m indoor to **30-50m+ outdoor**



Channels 11 to 25

THREAD SECURITY

- One network-wide symmetric key:
- Media Access Layer (MAC) authentication and encryption
 - Different levels of security as defined in IEEE 802.15.4
 - Does not change very often 👎

LEVEL	SECURITY LEVEL	DESCRIPTION	MIC* SIZE
0x00	None	No security (unencrypted)	-
0x01	MIC-32	Data authenticity only	32 bits
0x02	MIC-64	Data authenticity only	64 bits
0x03	MIC-128	Data authenticity only	128 bits
0x04	ENC	Confidentiality only	-
0x05	ENC-MIC-32	Confidentiality + Authenticity	32 bits
0x06	ENC-MIC-64	Confidentiality + Authenticity	64 bits
0x07	ENC-MIC-128	Confidentiality + Authenticity	128 bits



Application security

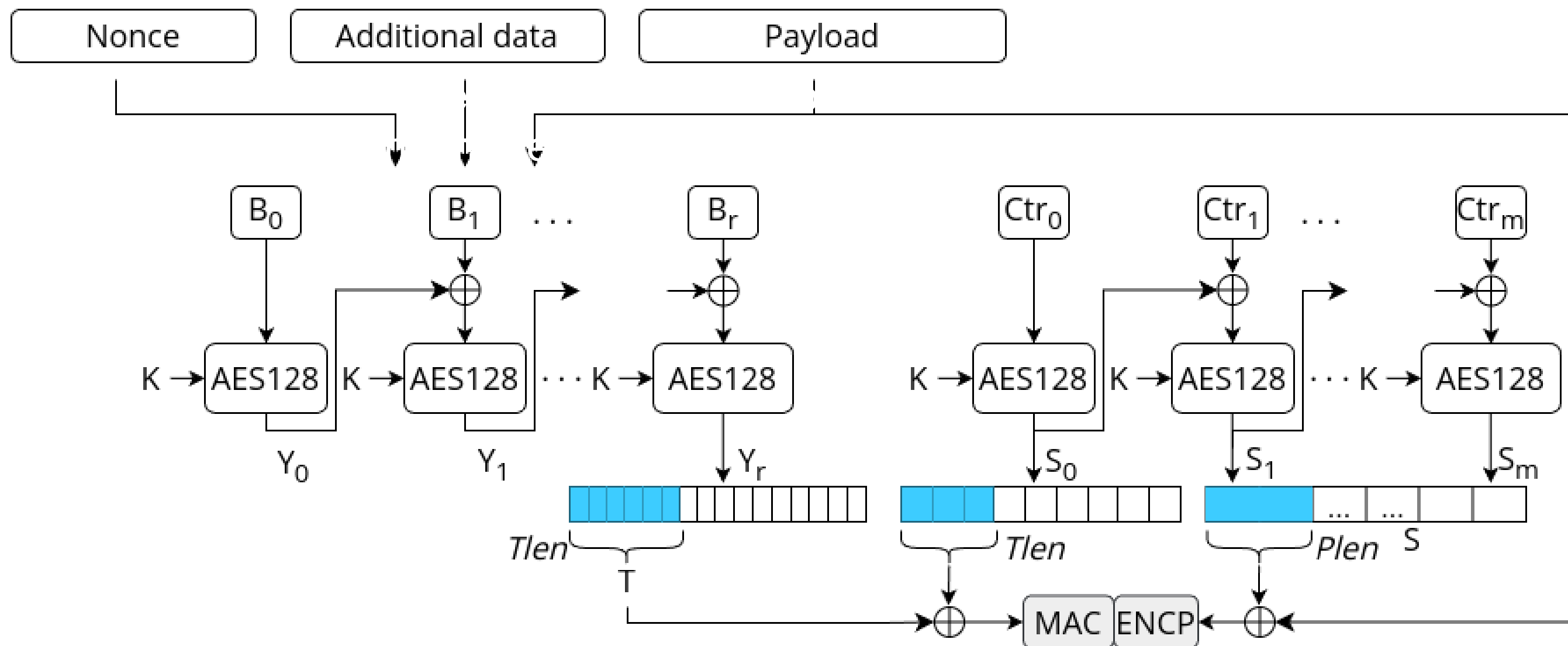
Thread security



*MIC: Message Integrity Code

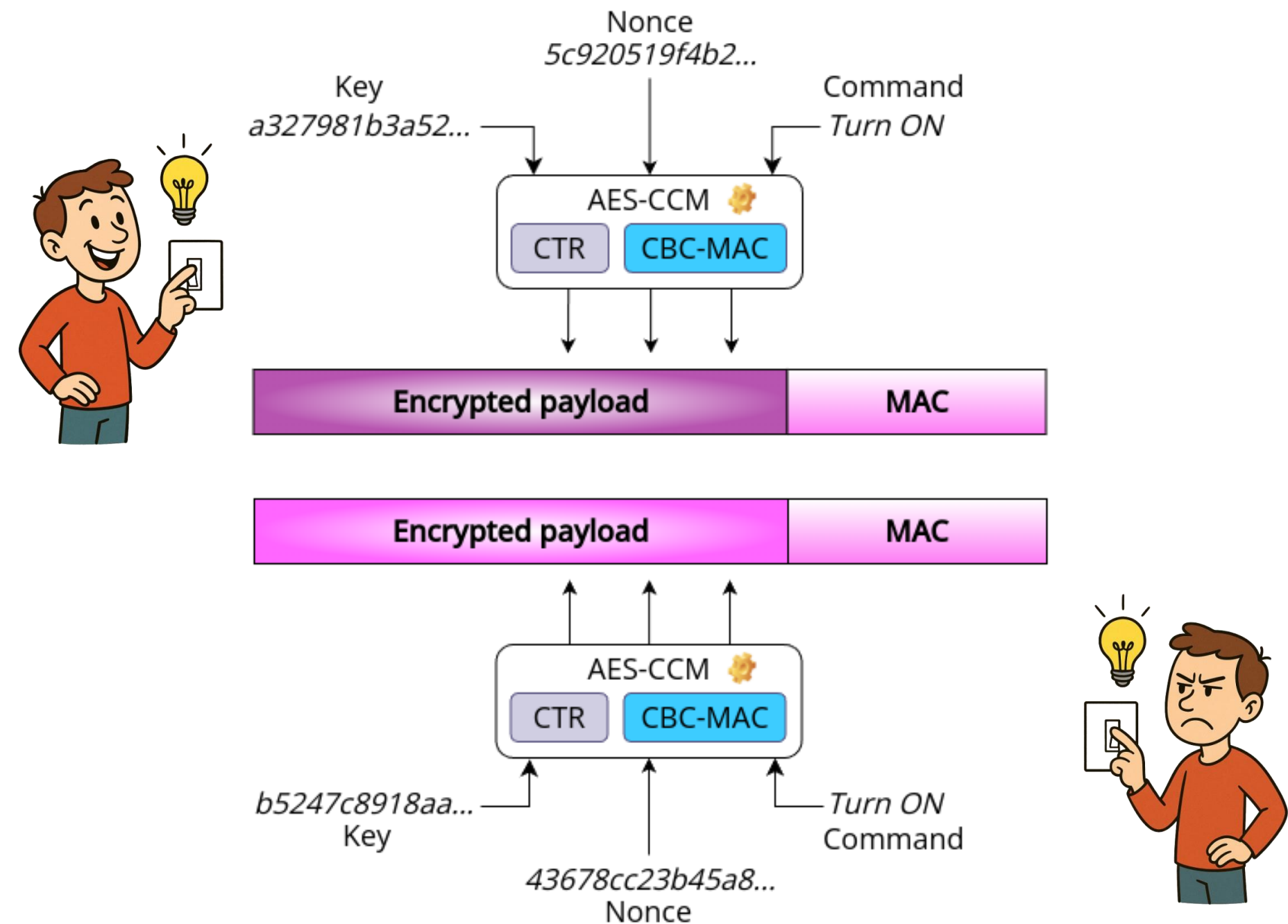
THREAD FRAME ENCRYPTION

- Uses the **AES-CCM** scheme to encrypt frames at the IEEE 802.15.4 (MAC/network) layer
- AES-CCM scheme (more specifically AES-CCM*), as defined by NIST 800-38C

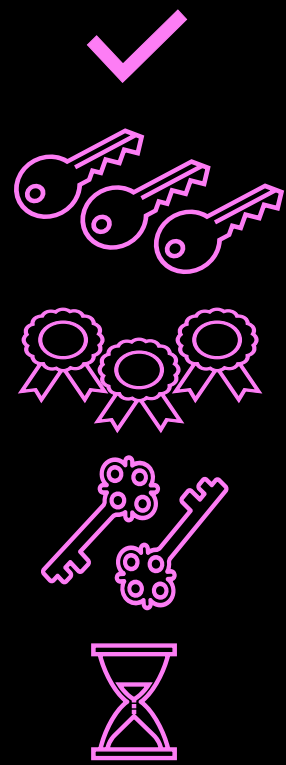


ESSENTIALLY

The same action (e.g., command) will generate the **same request ... response** packet sizes



WHAT ABOUT THE APPLICATION-LAYER?



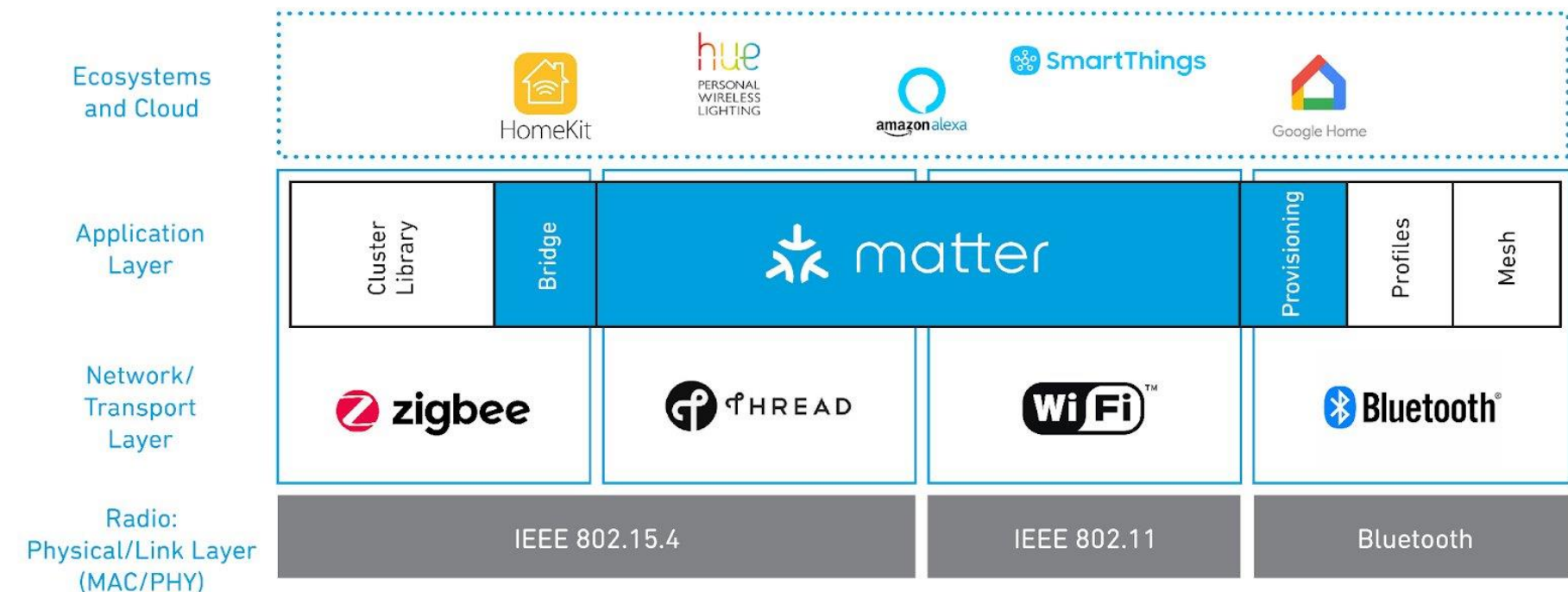
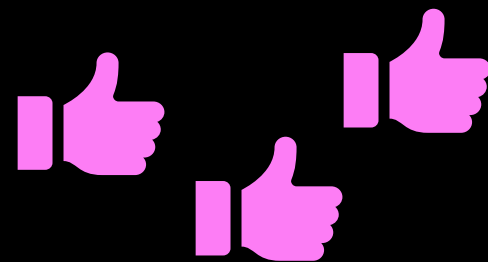
Mandatory security

Symmetric session keys

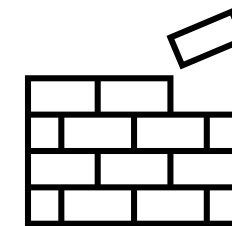
Fabric-specific certificates

Key exchange algorithms

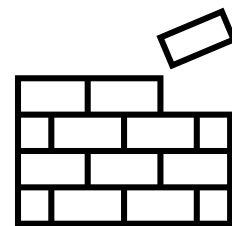
Ephemeral asymmetric keys



Source: <https://www.qorvo.com/design-hub/blog/matter-gets-everybody-talking>



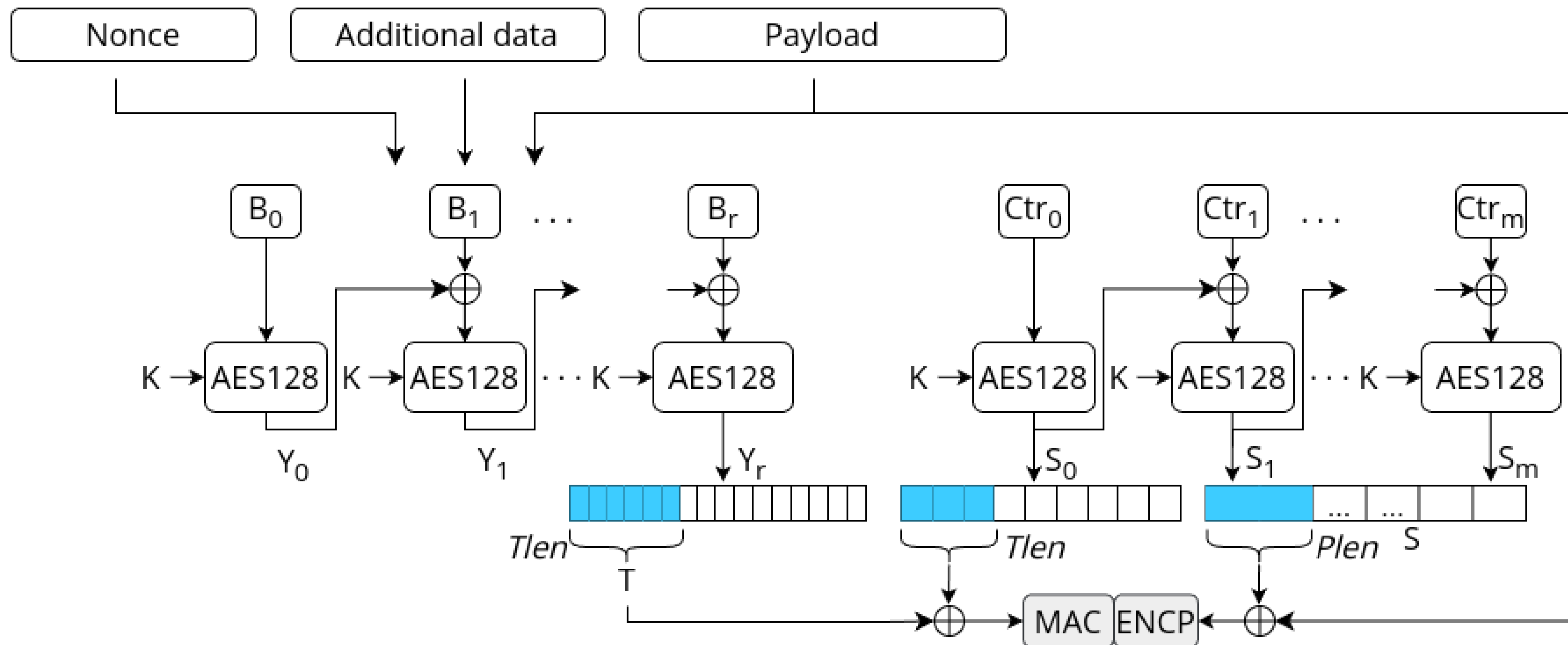
Interoperability



Mandatory security

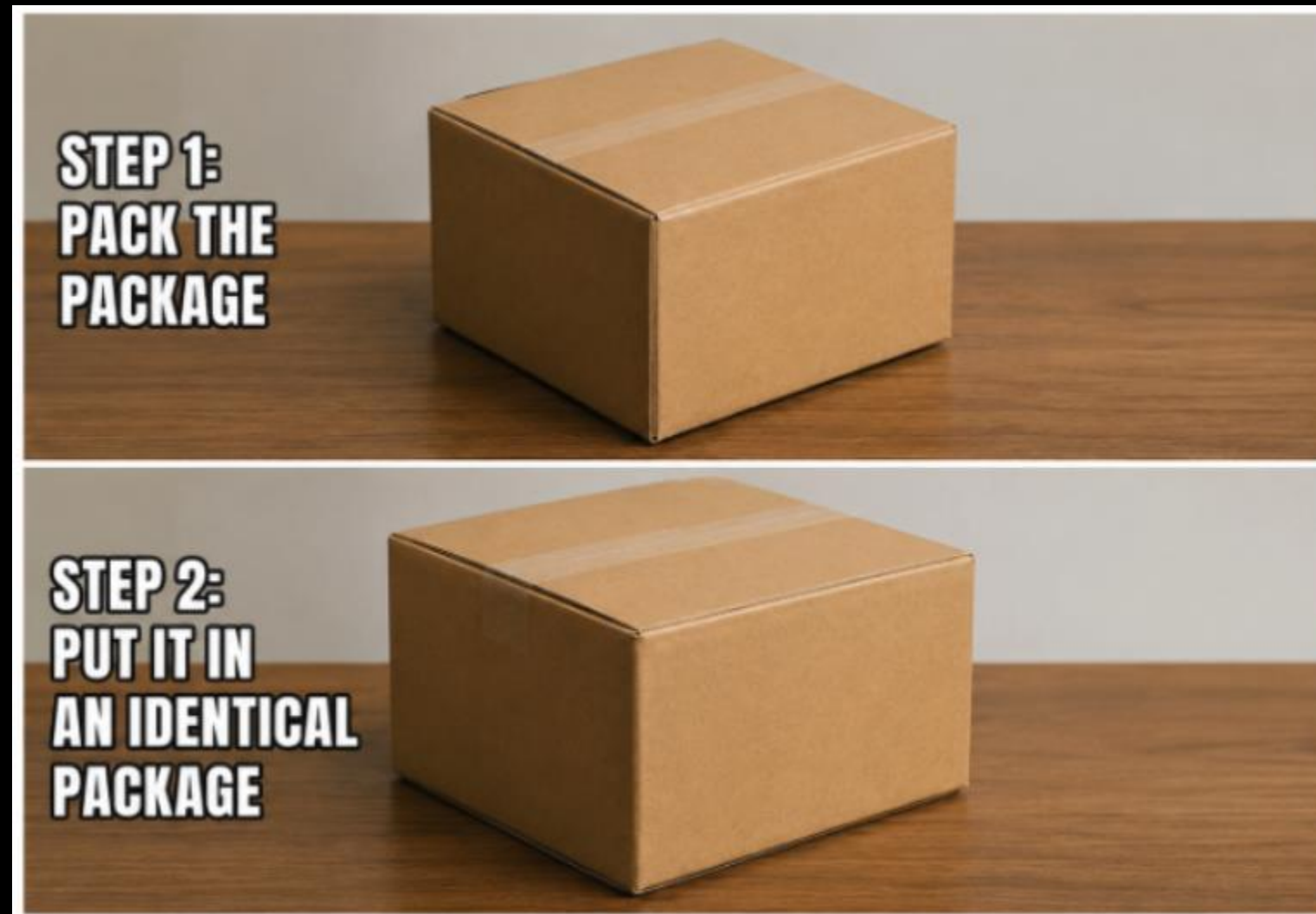
MATTER PACKET ENCRYPTION

- Matter uses the **AES-CCM** scheme to encrypt packets at the IEEE 802.15.4 (MAC/network) layer
- AES-CCM scheme, as defined by NIST 800-38C





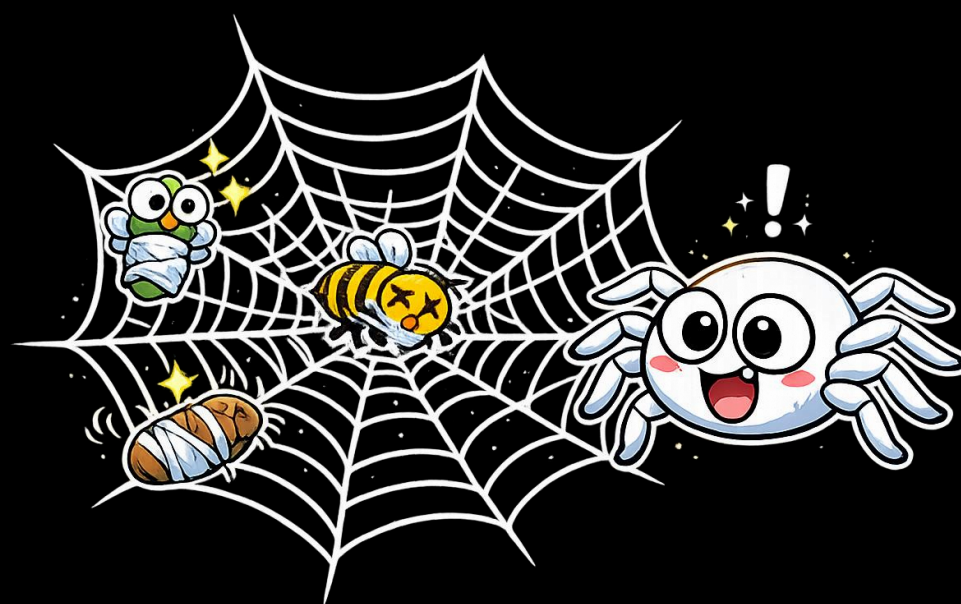
WHAT DOES THIS MEAN?



SO EVERYTHING IS (DOUBLE)
ENCRYPTED?



INFRASTRUCTURE DISCOVERY

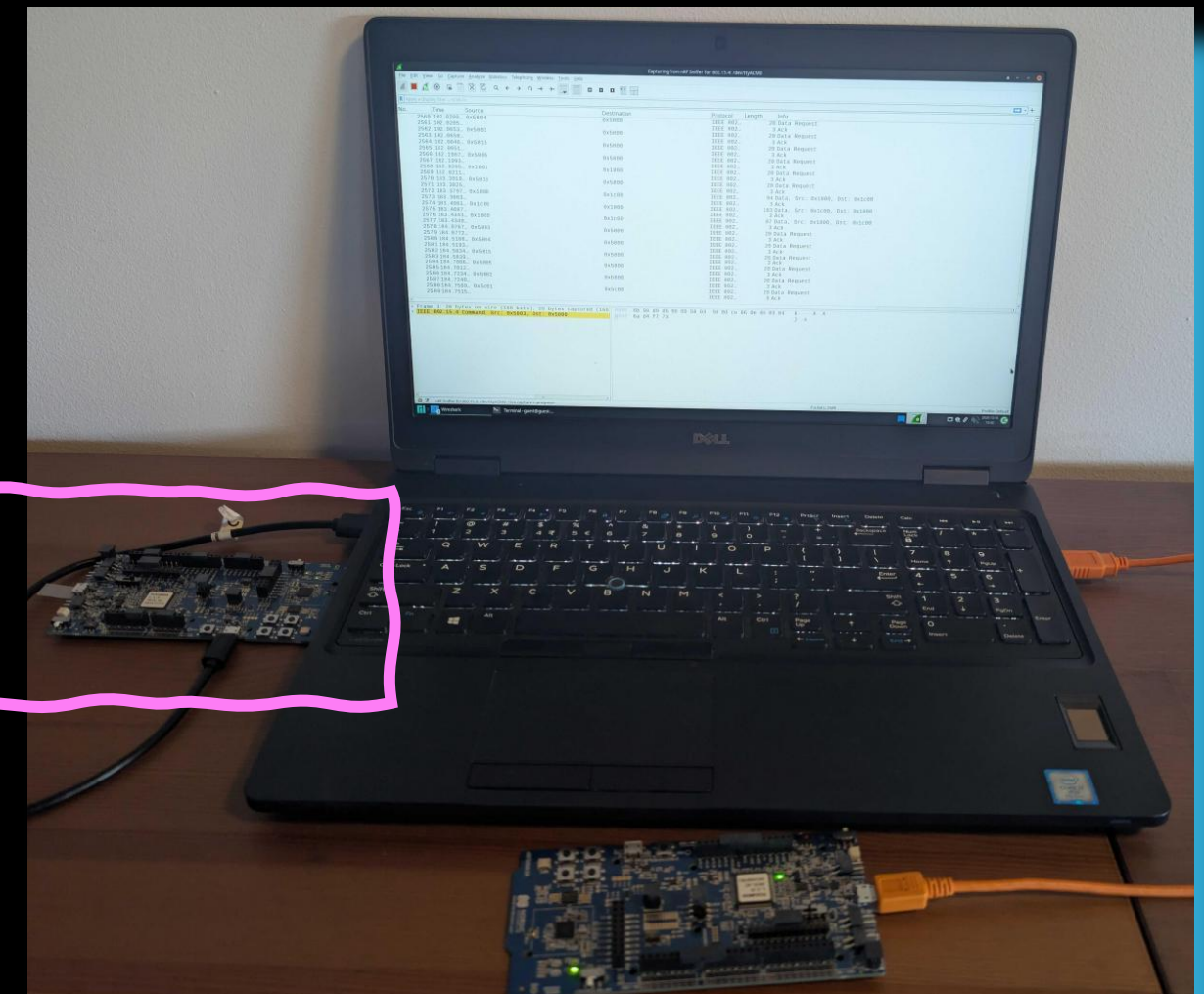
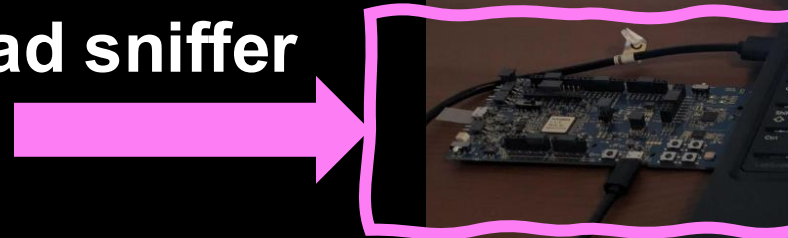


SNIFFING THREAD TRAFFIC

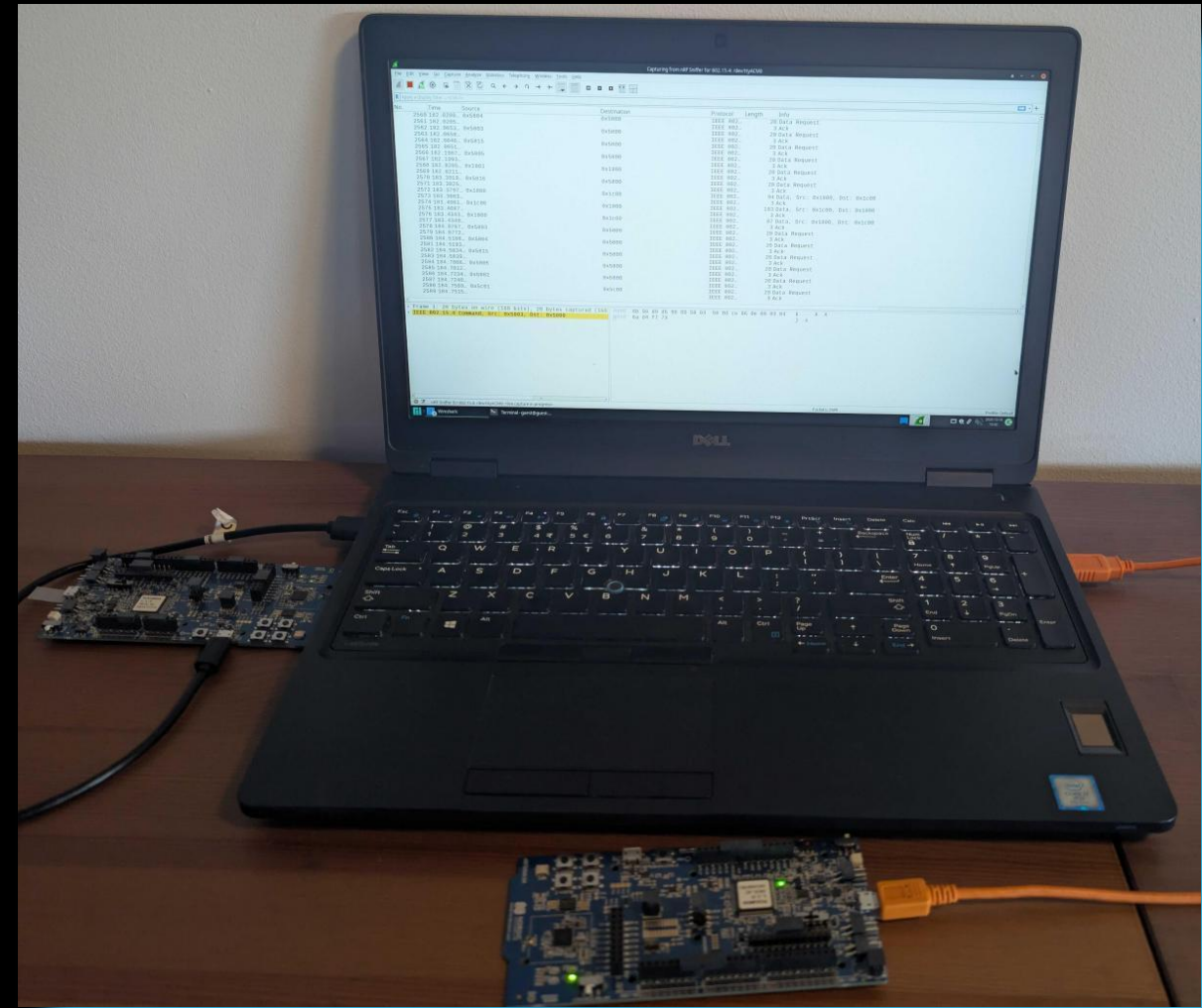
- 1 Get a Nordic nRF52840 DK development board
- 2 Download and install Nordic SDK
- 3 Compile & flash: **Thread sniffer**
- 4 Add Wireshark plug-in



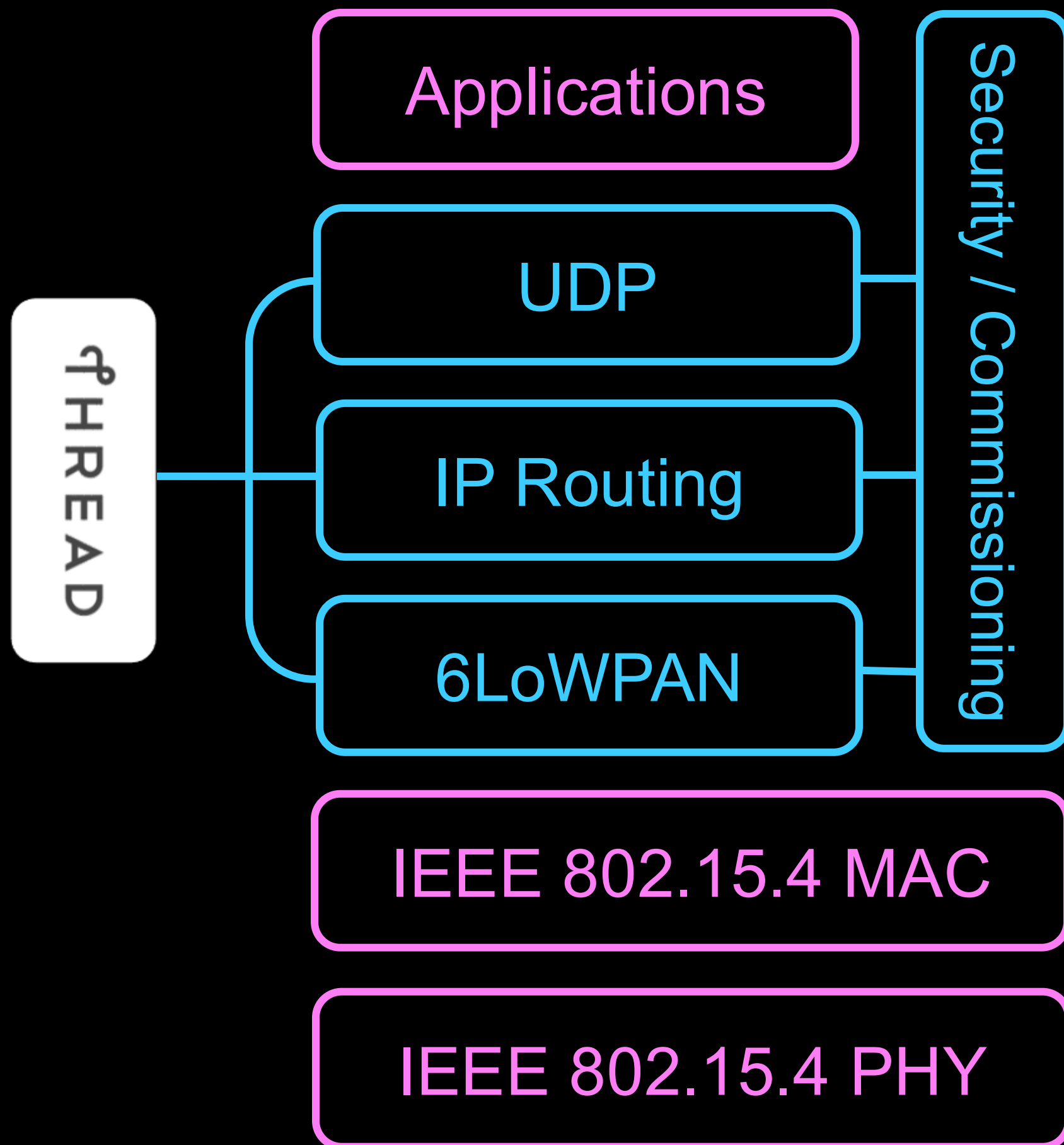
Thread sniffer



WHERE WE STARTED



WORK OBJECT: THE FRAME



Frames are the individual messages devices exchange over the network

Each frame is made up of protocol headers plus a payload

```
> Frame 445: Packet, 87 bytes on wire (696 bits), 87 bytes captured (696 bits) on interface /dev/ttyACM1, id 0
> IEEE 802.15.4 Data, Src: 0x6c00, Dst: 0xac00
> 6LoWPAN, Src: ::79d2:6940:cb50:459d, Dest: fda0:13ed:8be5::be9
> Internet Protocol Version 6, Src: ::79d2:6940:cb50:459d, Dst: fda0:13ed:8be5::be9
> User Datagram Protocol, Src Port: 5540, Dst Port: 60375
> Matter
```

(NON-ENCRYPTED) FIELDS OF INTEREST

```

- Frame 97: Packet, 103 bytes on wire (824 bits), 103 bytes captured (824 bits)
  - Section number: 1
  > Interface id: 0 (/dev/ttyACM1)
  - Encapsulation type: IEEE 802.15.4 Wireless PAN with FCS not present (127)
  - Arrival Time: Jan 15, 2026 14:23:12.734628000 EET
  - UTC Arrival Time: Jan 15, 2026 12:23:12.734628000 UTC
  - Epoch Arrival Time: 1768479792.734628000
  [Time shift for this packet: 0.000000000 seconds]
  - [Time delta from previous captured frame: 28.505000 milliseconds]
  - [Time delta from previous displayed frame: 28.505000 milliseconds]
  - [Time since reference or first frame: 10.323112000 seconds]
  - Frame Number: 97
  - Frame Length: 103 bytes (824 bits)
  - Capture Length: 103 bytes (824 bits)
  - [Frame is marked: False]
  - [Frame is ignored: False]
  - [Protocols in frame: wpan:data]
  - Character encoding: ASCII (0)
  - IEEE 802.15.4 Data, Src: 0xac00, Dst: 0x6c00
    - Frame Control Field: 0x9869, Frame Type: Data, Security Enabled, Acknowledged
    - Sequence Number: 96
    - Destination PAN: 0x90d5
    - Destination: 0x6c00
    - Source: 0xac00
  > Auxiliary Security Header
  - MIC: 578450da
  > [Expert Info (Warning/Undecoded): No encryption key set - can't decrypt]
  - Data (84 bytes)
    - Data: 06b1a5434d1ae869a8469ed792039f6dc19b3585192a9b8ebb51517613f0e3467e7e90
    [Length: 84]
```

- Epoch Arrival Time: 1768479792.734628000

- Frame Length: 103 bytes (824 bits)

- Frame Control Field: 0x9869, Frame Type: Data, Security Enabled, Acknowledged

- Sequence Number: 96

- Destination: 0x6c00

- Source: 0xac00



REMEMBER OUR PRIOR RESEARCH

**black hat**
BRIEFINGS

DECEMBER 10-11, 2025
EXCEL LONDON / UNITED KINGDOM



Ghosts in the Stream: Exposing Lives and Devices Behind Encrypted Doors

Speakers: Kristopher Schlett, Béla Genge
Contributors: Ioan Pădurean, Savio Sciancalepore

#BHEU @BlackHatEvents

In our initial attempt we simply used our approaches developed in the previous research

Matter	137 35879 → 5540	Len=75
Matter	130 5540 → 35879	Len=68
Matter	96 35879 → 5540	Len=34



Matter	137 35879 → 5540	Len=75
Matter	130 5540 → 35879	Len=68
Matter	96 35879 → 5540	Len=34

Matter	137 35879 → 5540	Len=75
Matter	130 5540 → 35879	Len=68
Matter	96 35879 → 5540	Len=34



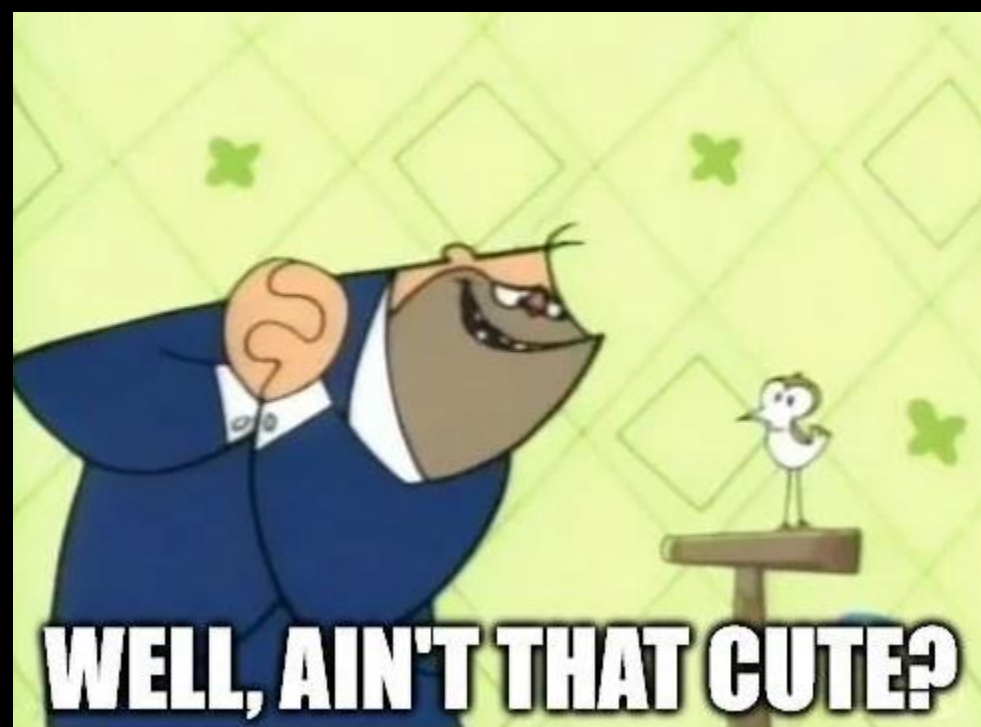
WE CAN JUST USE OUR PREVIOUS RESEARCH

```
Matter      96 5540 → 59065 Len=34
Matter     135 5540 → 59065 Len=73
Matter     104 59065 → 5540 Len=42
Matter      96 5540 → 59065 Len=34
Matter     121 59065 → 5540 Len=59
Matter     129 5540 → 59065 Len=67
Matter      96 59065 → 5540 Len=34
Matter     135 5540 → 59065 Len=73
Matter     104 59065 → 5540 Len=42
Matter      96 5540 → 59065 Len=34
Matter     121 59065 → 5540 Len=59
Matter     129 5540 → 59065 Len=67
Matter      96 59065 → 5540 Len=34
Matter     291 5540 → 59065 Len=229
Matter     104 59065 → 5540 Len=42
Matter      96 5540 → 59065 Len=34
Matter     262 5540 → 59065 Len=200
Matter     104 59065 → 5540 Len=42
Matter      96 5540 → 59065 Len=34
Matter     135 5540 → 59065 Len=73
Matter     104 59065 → 5540 Len=42
Matter      96 5540 → 59065 Len=34
Matter     135 5540 → 59065 Len=73
Matter     104 59065 → 5540 Len=42
```

One – to – one
mapping



```
IEEE 802.15.4 150 Data, Src: 0xb800, Dst: 0xa800
IEEE 802.15.4 31 Ack
IEEE 802.15.4 60 Data, Src: 0xb800, Dst: 0xa800
IEEE 802.15.4 131 Data, Src: 0xa800, Dst: 0xb800
IEEE 802.15.4 31 Ack
IEEE 802.15.4 31 Ack
IEEE 802.15.4 150 Data, Src: 0xb800, Dst: 0xa800
IEEE 802.15.4 60 Data, Src: 0xb800, Dst: 0xa800
IEEE 802.15.4 48 Data Request
IEEE 802.15.4 31 Ack
IEEE 802.15.4 127 Data, Src: 0xb800, Dst: 0xb804
IEEE 802.15.4 115 Data, Src: 0xb804, Dst: 0xb800
IEEE 802.15.4 31 Ack
IEEE 802.15.4 116 Data, Src: 0xb800, Dst: 0xa800
IEEE 802.15.4 31 Ack
IEEE 802.15.4 131 Data, Src: 0xa800, Dst: 0xa000
IEEE 802.15.4 31 Ack
IEEE 802.15.4 31 Ack
IEEE 802.15.4 31 Ack
IEEE 802.15.4 131 Data, Src: 0xa800, Dst: 0xa400
IEEE 802.15.4 31 Ack
IEEE 802.15.4 31 Ack
```

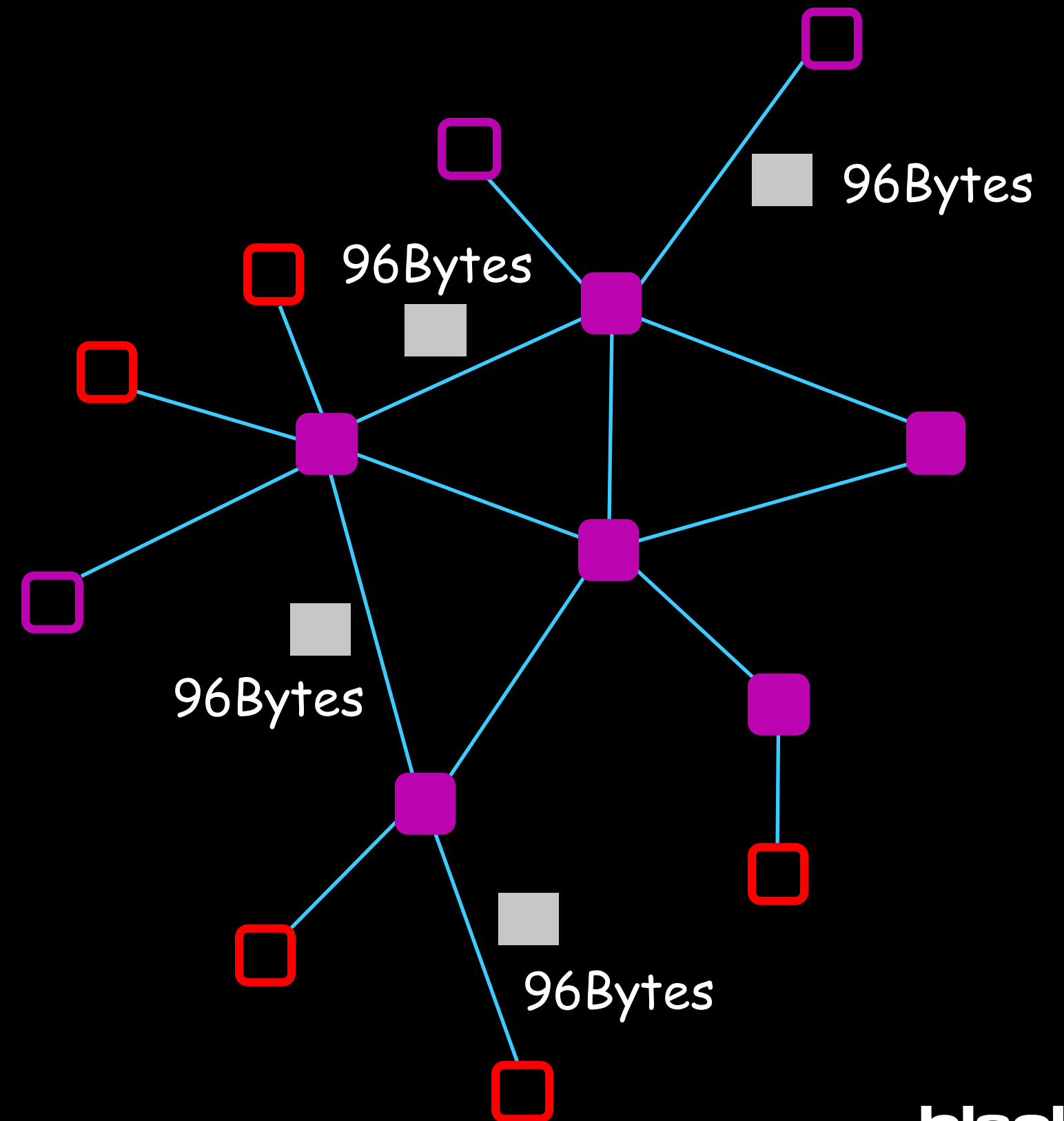


RESULT: VERY BAD – 37%



WHY WAS THIS THE CASE?

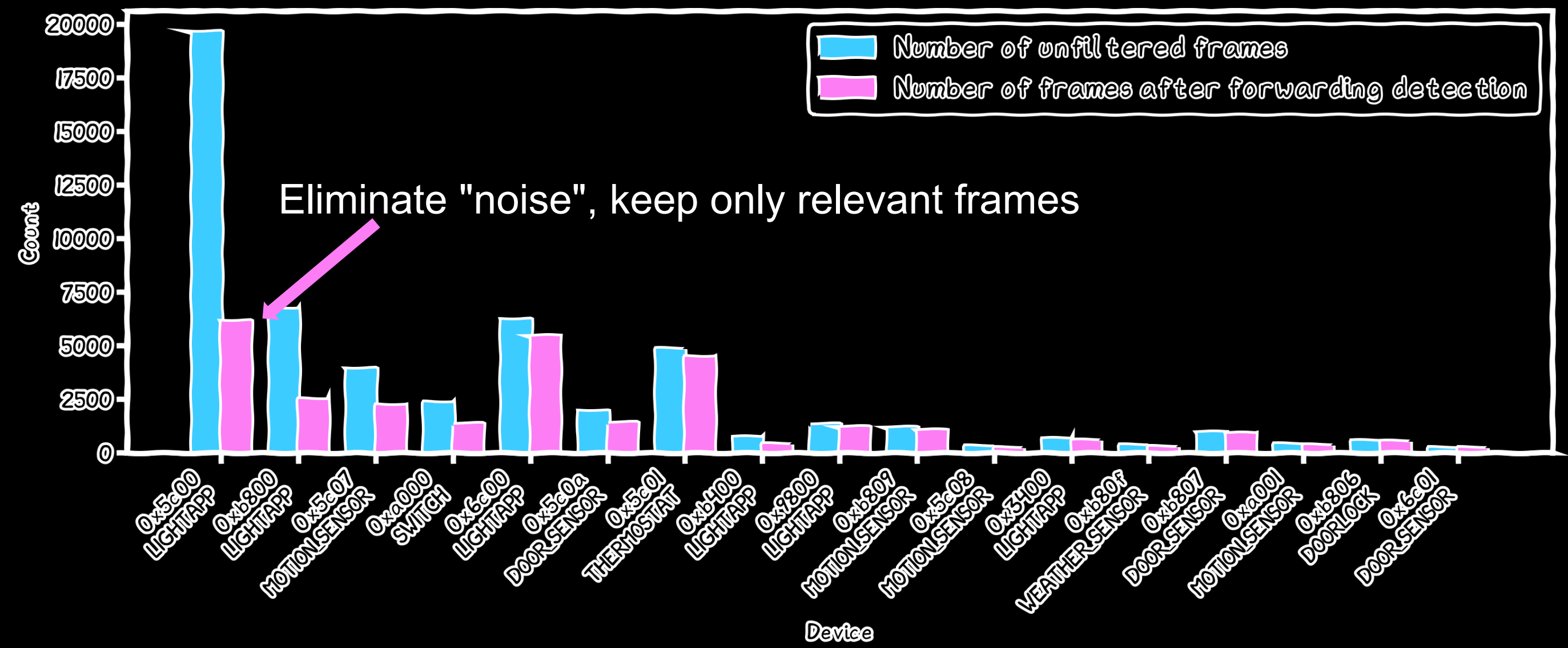
- When forwarded through the mesh network, the same frame shows up several times
- The sniffer picks up the same frame several times
- Solution: identify **unique frames** (forwarding detection)



HEURISTICS FOR FORWARDING DETECTION

```
if recv(frame) and send(frame) and delta(recv, send) < Threshold then
    mark_forwarded(frame)
end_if
```

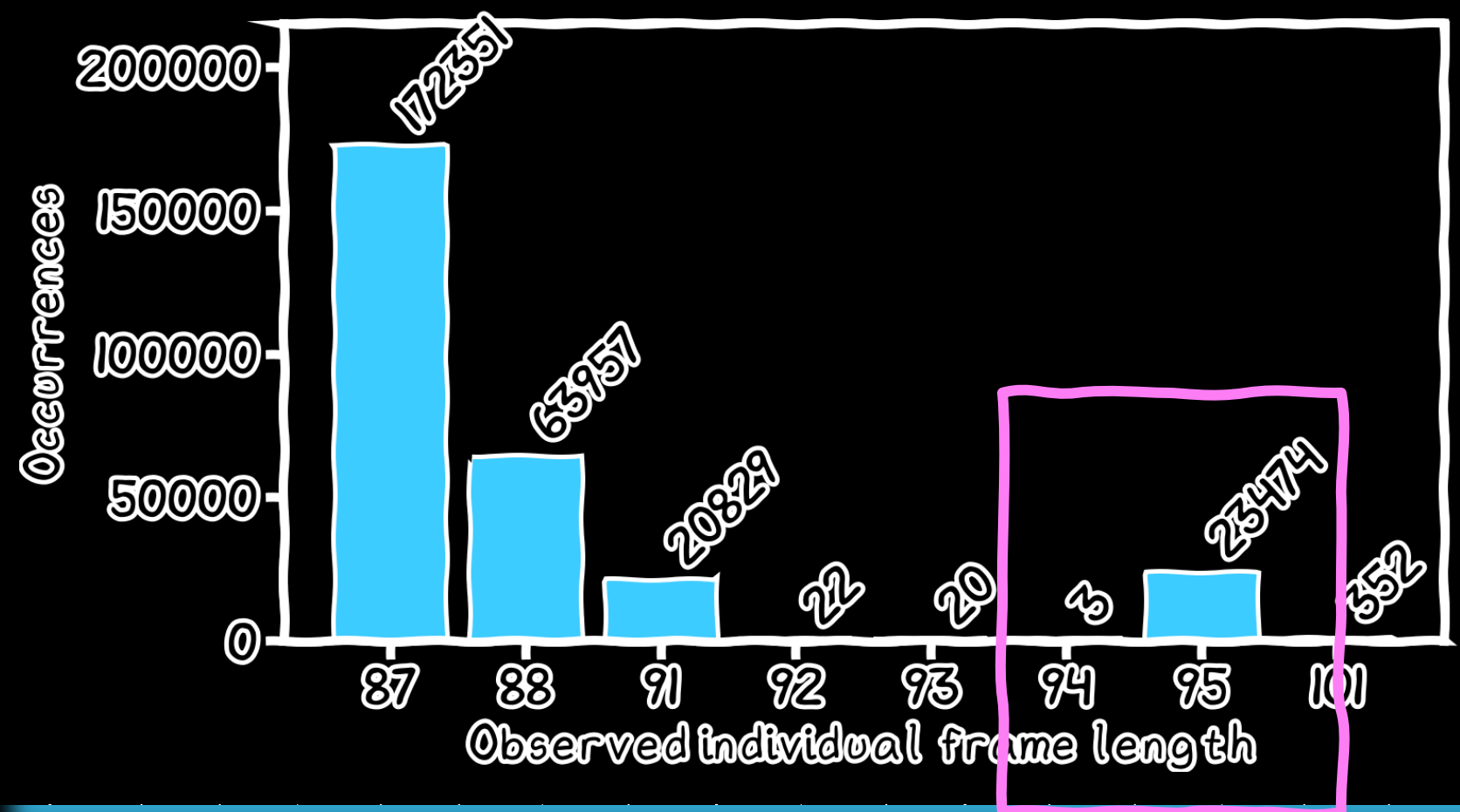
Value of Threshold determined empirically (~0.8s)



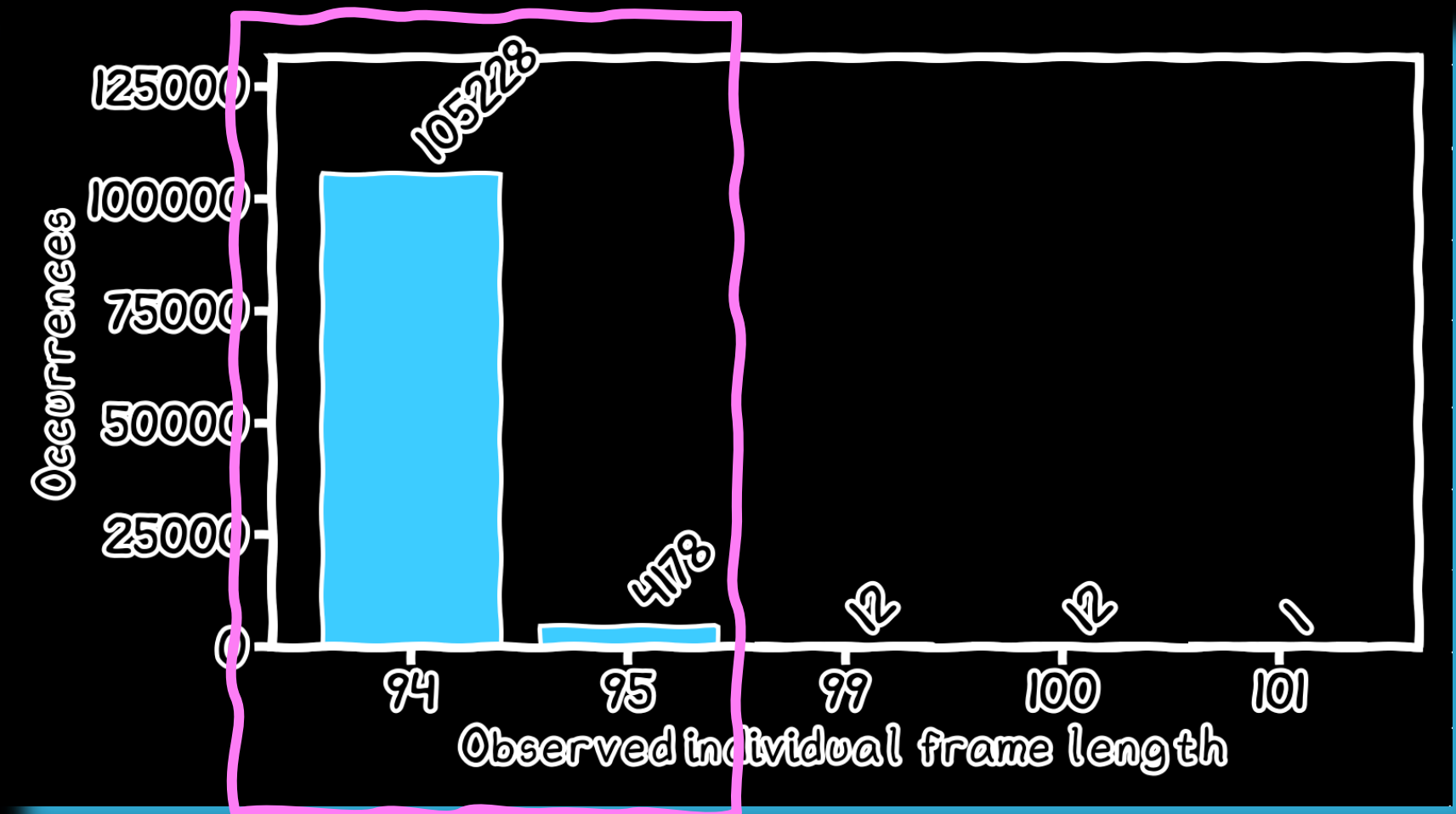
LETS TRY AGAIN

RESULT: BETTER – 44%, STILL NOT GOOD

WHY WAS THIS THE CASE?

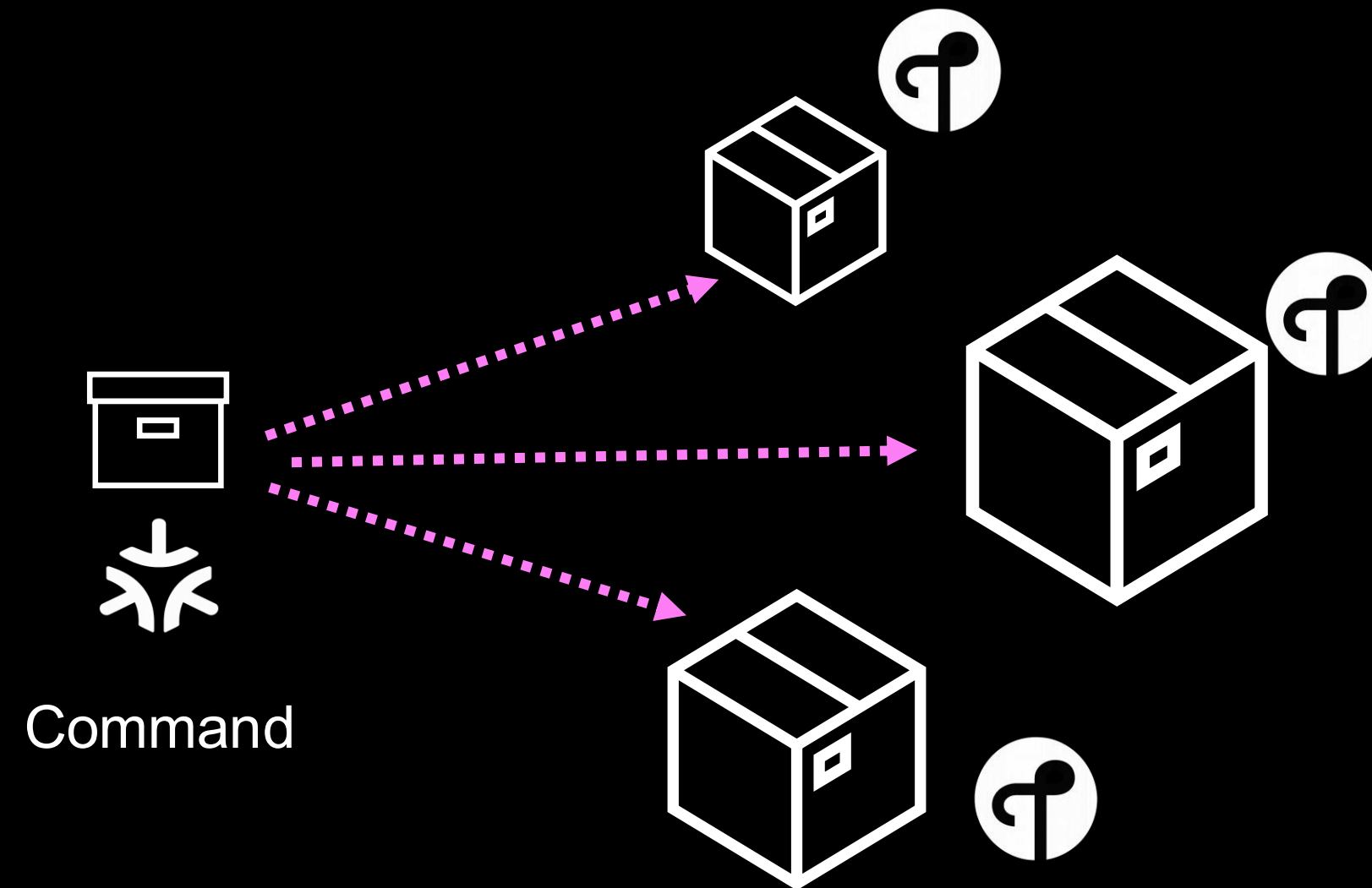


← Payload of 42 bytes



FRAME TRANSMISSION: DIFFERENT SIZES

The same **Matter** command can be packaged in **Thread** frames of different sizes



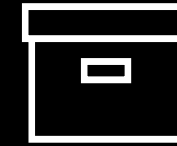
WHAT EXACTLY IS HAPPENING?

Nothing at the application layer seems different between commands packaged in larger or smaller frames

1

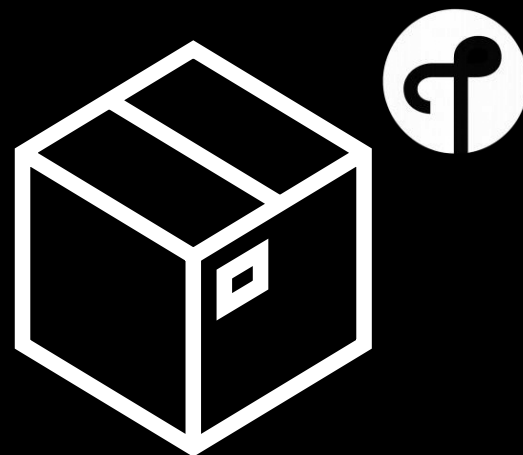


Command

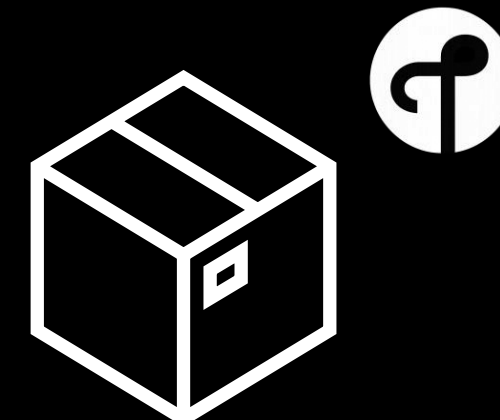


2

```
✓ Matter
  > Message Flags: 0x00, Destination ID Type: Not present
  | Session ID: 0xfc77
  > Security Flags: 0x00, Session Type: Unicast Session
  | Message Counter: 38695646
  | Encrypted Payload (10 bytes)
  | Integrity Check: 9ba27d91dd58c27d20f16f87223b702a
```



```
✓ Matter
  > Message Flags: 0x00, Destination ID Type: Not present
  | Session ID: 0xacee
  > Security Flags: 0x00, Session Type: Unicast Session
  | Message Counter: 194035104
  | Encrypted Payload (10 bytes)
  | Integrity Check: 47772b688b88ce0886bb36afc63893b9
```



WHY IS THIS HAPPENING?

6LoWPAN, Src: fda0:13ed:8be5::be9, Dest: ::2d14:840c:e1a2:b52c

Fragmentation Header

IPHC Header

011. = Pattern: IP header compression (0x03)

...0 1... = Traffic class and flow label: ECN and flow label inline (0x1)

.... .1... = Next header: Compressed

.... ..00 = Hop limit: Inline (0x0)

.... 1... = Context identifier extension: True

....0.. = Source address compression: Stateless

....00 = Source address mode: Inline (0x0000)

.... 0... = Multicast address compression: False

....1.. = Destination address compression: Stateful

....01 = Destination address mode: 64-bits inline (0x0001)

0000 = Source context identifier: 0x0

.... 0001 = Destination context identifier: 0x1

00.. = ECN: 0

1

- Hop limit (decreases after each "hop"):
- 1, 64, 255: compressed
- other values: **inline** takes up space

Some nodes add Fragmentation Header (fragment of 1)

Traffic class, flow label, etc.: inline / compressed

6LoWPAN, Src: ::a2c5:f62:1ac0:80f0, Dest: fda0:13ed:8be5::be9

IPHC Header

011. = Pattern: IP header compression (0x03)

...1 1... = Traffic class and flow label: Version, traffic class, and flow label compressed (0x3)

.... .1... = Next header: Compressed

.... ..10 = Hop limit: 64 (0x2)

.... 1... = Context identifier extension: True

....1.. = Source address compression: Stateful

....01 = Source address mode: 64-bits inline (0x0001)

.... 0... = Multicast address compression: False

....0.. = Destination address compression: Stateless

....00 = Destination address mode: Inline (0x0000)

0001 = Source context identifier: 0x1

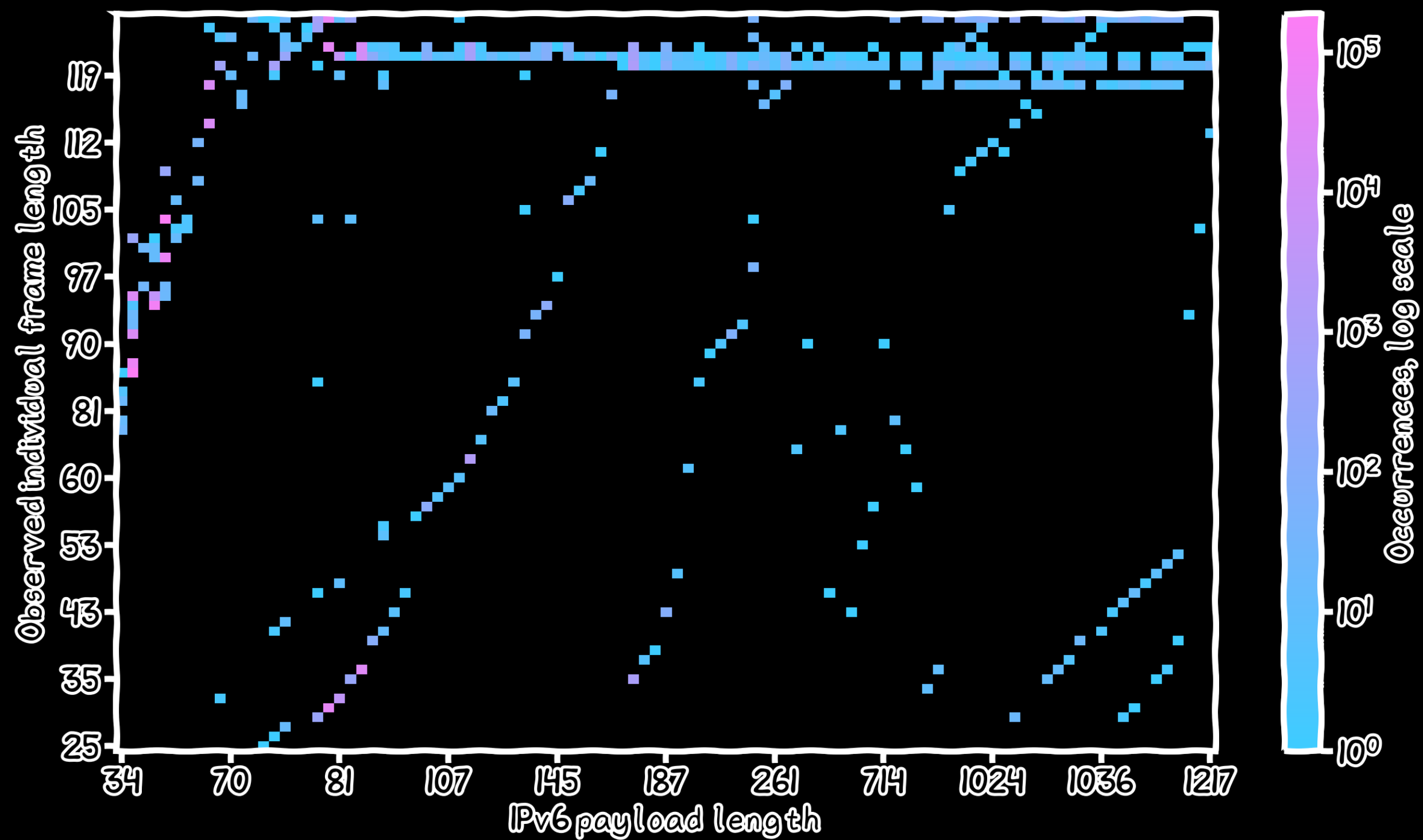
.... 0000 = Destination context identifier: 0x0

Source: ::a2c5:f62:1ac0:80f0

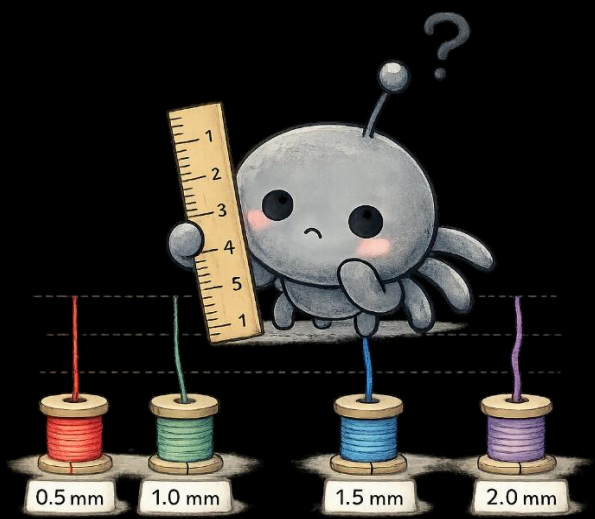
Destination: fda0:13ed:8be5::be9

2

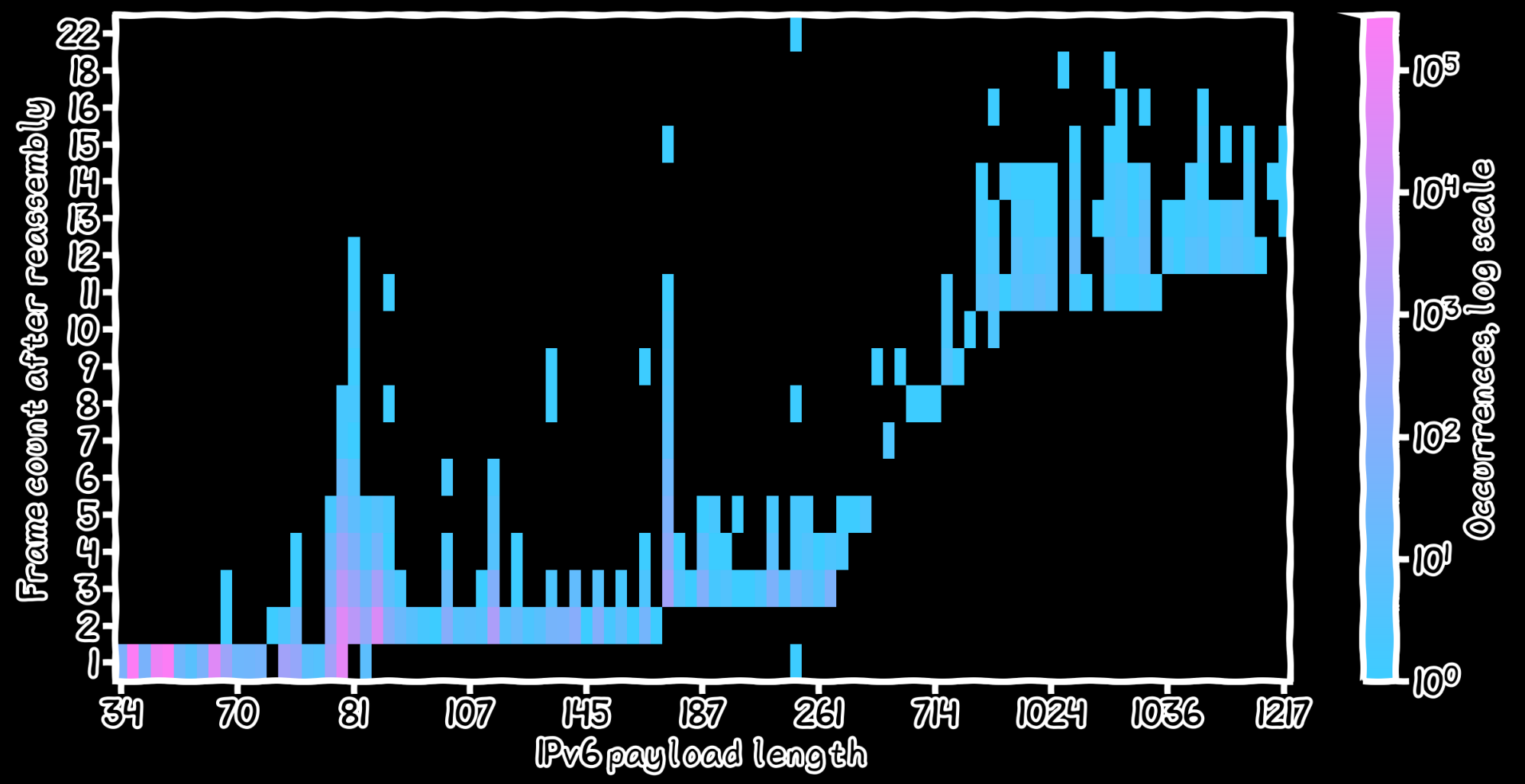
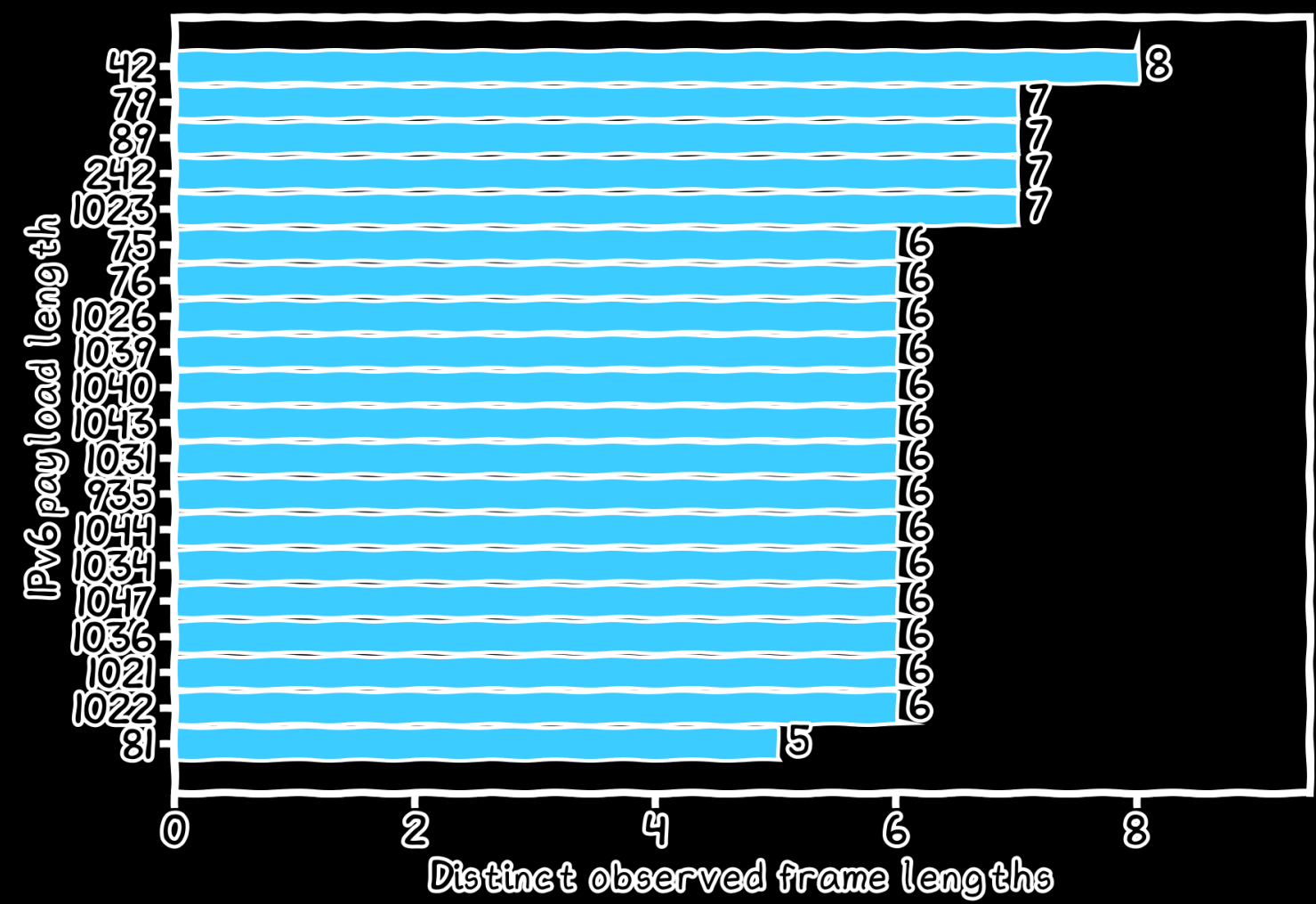
FRAME SIZE DISTRIBUTION



The same payload transmitted in frames of different length

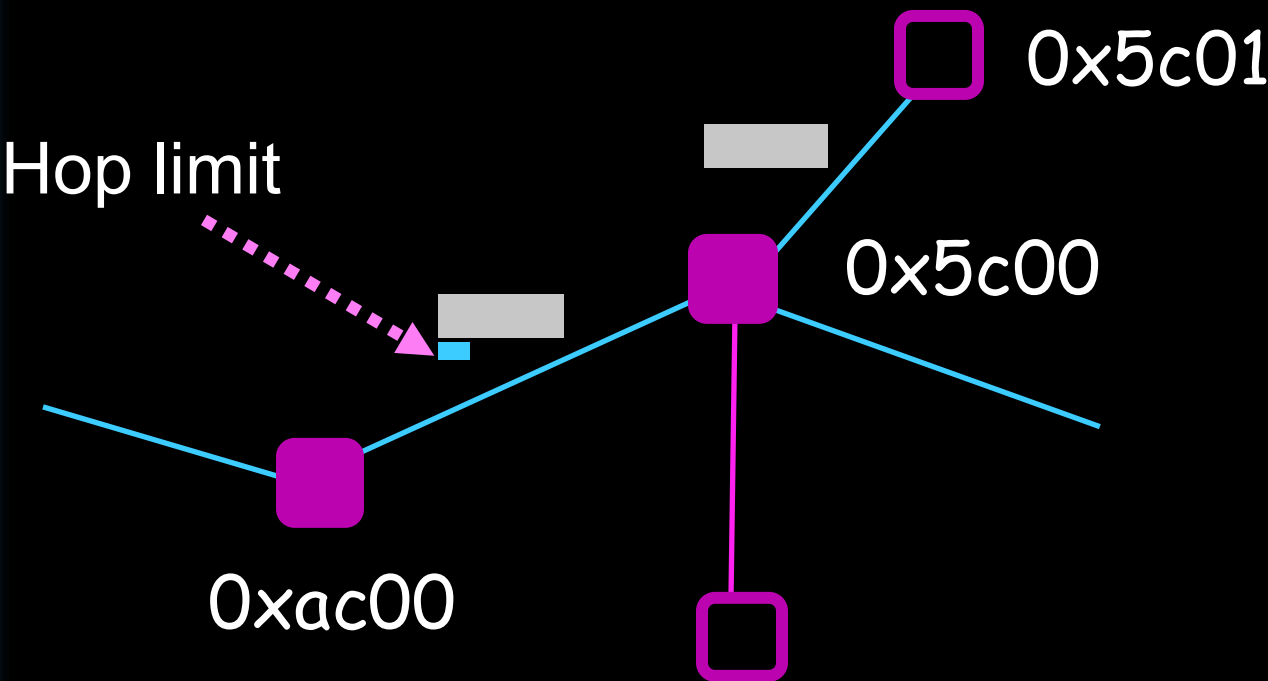


REALITY IS COMPLEX



FRAMES CHANGE: HOP LIMIT

```
> IEEE 802.15.4 Data, Src: 0x5c01, Dst: 0x5c00
└─ 6LoWPAN, Src: ::a2c5:f62:1ac0:80f0, Dest: fda0:13ed:8be5::be9
  └─ IPHC Header
    └─ 011. .... = Pattern: IP header compression (0x03)
    └─ ...1 1... .... = Traffic class and flow label: Version, traffic class, and flow label compressed (0x3)
    └─ .... .1.. .... = Next header: Compressed
    └─ .... ..10 .... = Hop limit: 64 (0x2)
    └─ .... .... 1... .... = Context identifier extension: True
    └─ .... .... .1.. .... = Source address compression: Stateful
    └─ .... .... ..01 .... = Source address mode: 64-bits inline (0x0001)
    └─ .... .... .... 0... = Multicast address compression: False
    └─ .... .... .... .0.. = Destination address compression: Stateless
    └─ .... .... .... ..00 = Destination address mode: Inline (0x0000)
    └─ 0001 .... = Source context identifier: 0x1
    └─ .... 0000 = Destination context identifier: 0x0
```



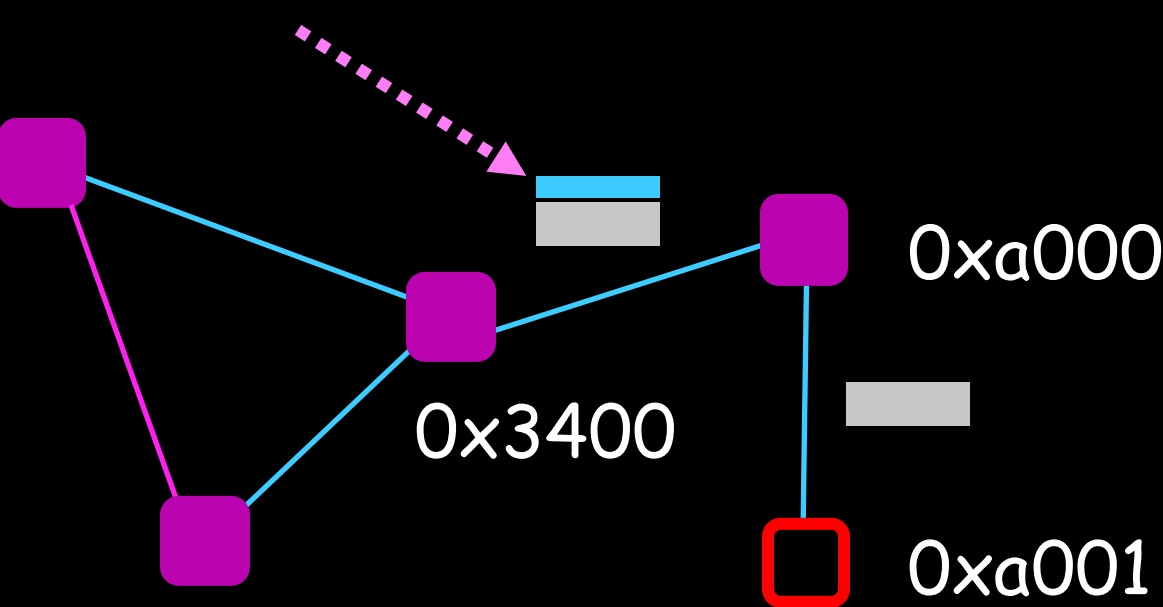
Hop limit changes

```
> IEEE 802.15.4 Data, Src: 0x5c00, Dst: 0xac00
└─ 6LoWPAN, Src: ::a2c5:f62:1ac0:80f0, Dest: fda0:13ed:8be5::be9
  └─ IPHC Header
    └─ 011. .... = Pattern: IP header compression (0x03)
    └─ ...1 1... .... = Traffic class and flow label: Version, traffic class, and flow label compressed (0x3)
    └─ .... .1.. .... = Next header: Compressed
    └─ .... ..00 .... = Hop limit: Inline (0x0)
    └─ .... .... 1... .... = Context identifier extension: True
    └─ .... .... .1.. .... = Source address compression: Stateful
    └─ .... .... ..01 .... = Source address mode: 64-bits inline (0x0001)
    └─ .... .... .... 0... = Multicast address compression: False
    └─ .... .... .... .0.. = Destination address compression: Stateless
    └─ .... .... .... ..00 = Destination address mode: Inline (0x0000)
    └─ 0001 .... = Source context identifier: 0x1
    └─ .... 0000 = Destination context identifier: 0x0
  └─ Hop limit: 63
```

FRAMES CHANGE: MESH HEADER

```
> IEEE 802.15.4 Data, Src: 0x3400, Dst: 0xa000
└─ 6LoWPAN, Src: fda0:13ed:8be5::be9, Dest: ::e9c7:64aa:1cf2:b5f5
  └─ Mesh Header
    └─ Flags
      └─ 10.. .... = Pattern: Mesh (0x02)
      └─ ..1. .... = V: True
      └─ ...1 .... = D: True
      └─ .... 1111 = Hops left: 15
      └─ Deep Hops left (Flags.Hops left == 15): 18
      └─ Originator: 0xac00
      └─ Destination: 0xa000
    > Fragmentation Header
    > IPHC Header
      └─ 011. .... = Pattern: IP header compression (0x03)
      └─ ...0 1... .... = Traffic class and flow label: ECN and flow label inline (0x1)
      └─ .... .1.. .... = Next header: Compressed
      └─ .... ..00 .... = Hop limit: Inline (0x0)
      └─ .... .... 1... .... = Context identifier extension: True
      └─ .... .... .0.. .... = Source address compression: Stateless
      └─ .... .... ..00 .... = Source address mode: Inline (0x0000)
      └─ .... .... .... 0... = Multicast address compression: False
      └─ .... .... .... .1.. = Destination address compression: Stateful
      └─ .... .... .... ..01 = Destination address mode: 64-bits inline (0x0001)
```

Mesh header

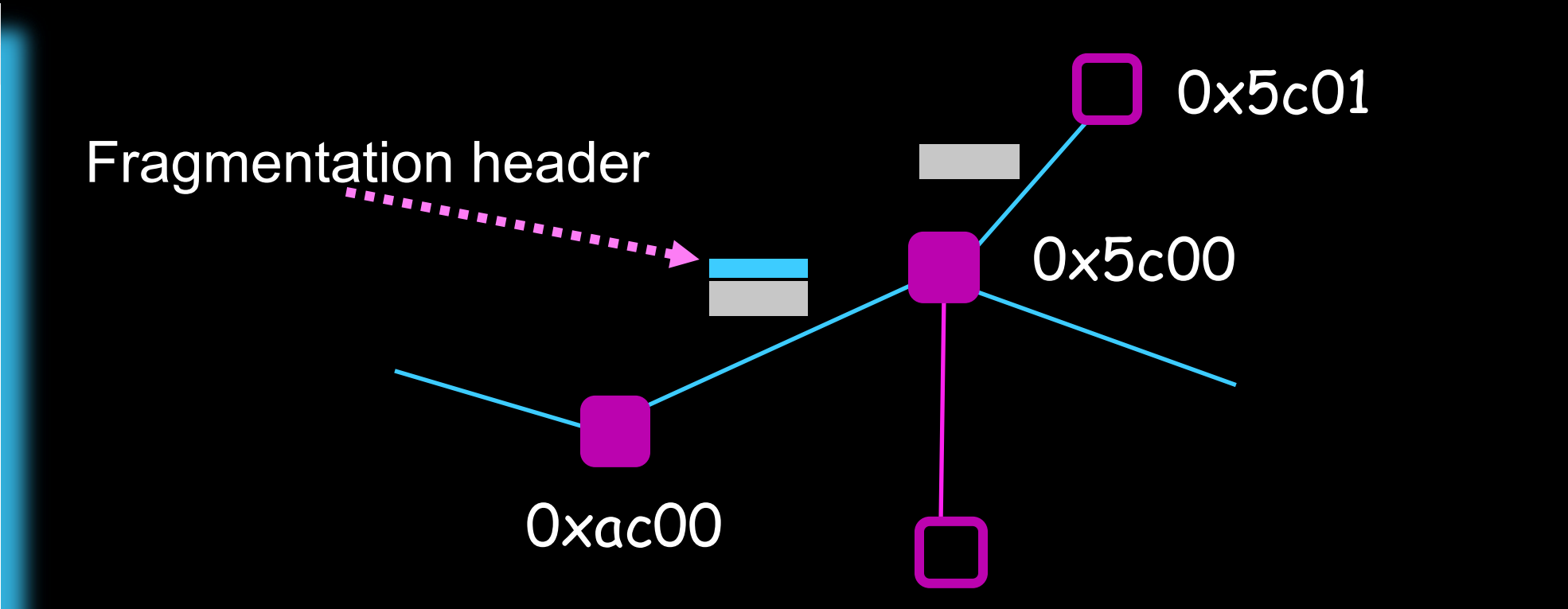


```
> IEEE 802.15.4 Data, Src: 0xa000, Dst: 0xa001
└─ 6LoWPAN, Src: fda0:13ed:8be5::be9, Dest: ::e9c7:64aa:1cf2:b5f5
  └─ IPHC Header
    └─ 011. .... = Pattern: IP header compression (0x03)
    └─ ...0 1... .... = Traffic class and flow label: ECN and flow label inline (0x1)
    └─ .... .1.. .... = Next header: Compressed
    └─ .... ..00 .... = Hop limit: Inline (0x0)
    └─ .... .... 1... .... = Context identifier extension: True
    └─ .... .... .0.. .... = Source address compression: Stateless
    └─ .... .... ..00 .... = Source address mode: Inline (0x0000)
    └─ .... .... .... 0... = Multicast address compression: False
    └─ .... .... .... .1.. = Destination address compression: Stateful
    └─ .... .... .... ..01 = Destination address mode: 64-bits inline (0x0001)
```

Mesh header is removed

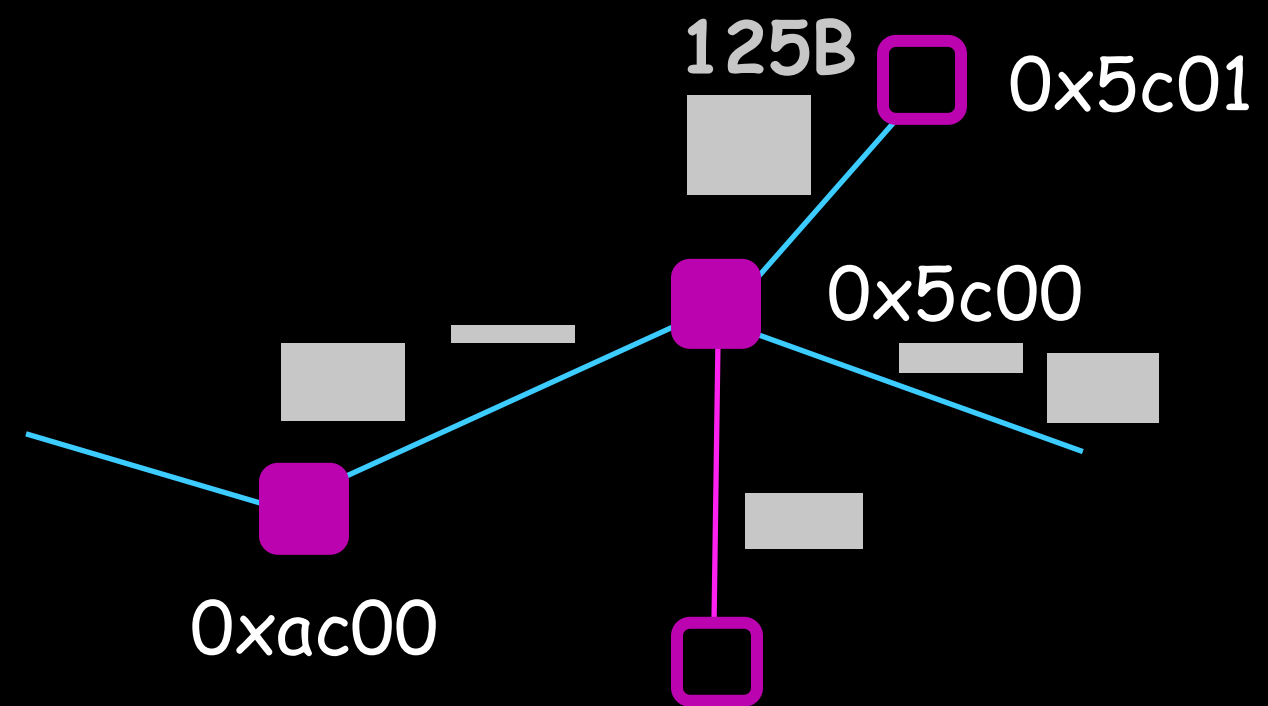
FRAMES CHANGE: FRAGMENTATION HEADER

```
> IEEE 802.15.4 Data, Src: 0xac00, Dst: 0x5c00
└─ 6LoWPAN, Src: fda0:13ed:8be5::be9, Dest: ::a2c5:f62:1ac0:80f0
  └─ Fragmentation Header
    └─ 1100 0... = Pattern: First fragment (0x18)
    └─ Datagram size: 90
    └─ Datagram tag: 0x1a14
  └─ IPHC Header
    └─ 011. .... = Pattern: IP header compression (0x03)
    └─ ...0 1... .... = Traffic class and flow label: ECN and flow label inline (0x1)
    └─ .... .1.. .... = Next header: Compressed
    └─ .... ..00 .... = Hop limit: Inline (0x0)
    └─ .... .... 1... .... = Context identifier extension: True
    └─ .... .... .0.. .... = Source address compression: Stateless
    └─ .... .... ..00 .... = Source address mode: Inline (0x0000)
    └─ .... .... .... 0... = Multicast address compression: False
    └─ .... .... .... .1.. = Destination address compression: Stateful
    └─ .... .... .... ..01 = Destination address mode: 64-bits inline (0x0001)
    └─ 0000 .... = Source context identifier: 0x0
    └─ .... 0001 = Destination context identifier: 0x1
    └─ 00.. .... = ECN: 0
    └─ ..00 .... = Padding: 0x00
    └─ .... 0100 1110 1000 1101 0001 = Flow label: 0x04e8d1
    └─ Hop limit: 63
```



Fragmentation header is removed

```
> IEEE 802.15.4 Data, Src: 0x5c00, Dst: 0x5c01
└─ 6LoWPAN, Src: fda0:13ed:8be5::be9, Dest: ::a2c5:f62:1ac0:80f0
  └─ IPHC Header
    └─ 011. .... = Pattern: IP header compression (0x03)
    └─ ...0 1... .... = Traffic class and flow label: ECN and flow label inline (0x1)
    └─ .... .1.. .... = Next header: Compressed
    └─ .... ..00 .... = Hop limit: Inline (0x0)
    └─ .... .... 1... .... = Context identifier extension: True
    └─ .... .... .0.. .... = Source address compression: Stateless
    └─ .... .... ..00 .... = Source address mode: Inline (0x0000)
    └─ .... .... .... 0... = Multicast address compression: False
    └─ .... .... .... .1.. = Destination address compression: Stateful
    └─ .... .... .... ..01 = Destination address mode: 64-bits inline (0x0001)
    └─ 0000 .... = Source context identifier: 0x0
    └─ .... 0001 = Destination context identifier: 0x1
    └─ 00.. .... = ECN: 0
    └─ ..00 .... = Padding: 0x00
    └─ .... 0100 1110 1000 1101 0001 = Flow label: 0x04e8d1
    └─ Hop limit: 62
```

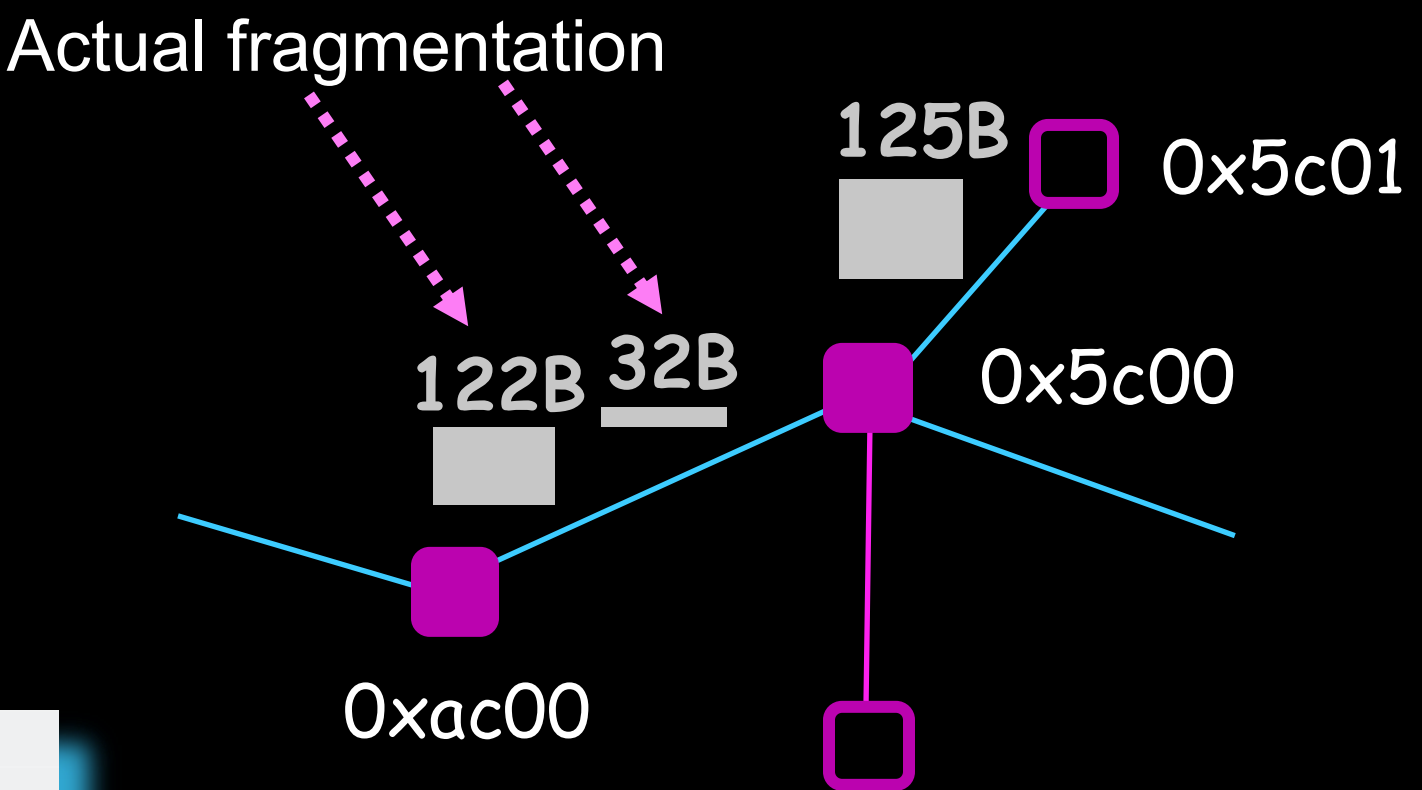


FRAMES CHANGE: SUDDEN FRAGMENTATION

```
> IEEE 802.15.4 Data, Src: 0x5c01, Dst: 0x5c00
└─ 6LoWPAN, Src: ::a2c5:f62:1ac0:80f0, Dest: fda0:13ed:8be5::be9
  └─ IPHC Header
    └─ 011. .... = Pattern: IP header compression (0x03)
    └─ ...1 1... .... = Traffic class and flow label: Version, traffic
    └─ .... .1.. .... = Next header: Compressed
    └─ .... ..10 .... = Hop limit: 64 (0x2)
    └─ .... .... 1... .... = Context identifier extension: True
    └─ .... .... .1.. .... = Source address compression: Stateful
    └─ .... .... ..01 .... = Source address mode: 64-bits inline (0x0001)
    └─ .... .... .... 0... = Multicast address compression: False
    └─ .... .... .... .0.. = Destination address compression: Stateless
    └─ .... .... .... ..00 = Destination address mode: Inline (0x0000)
```

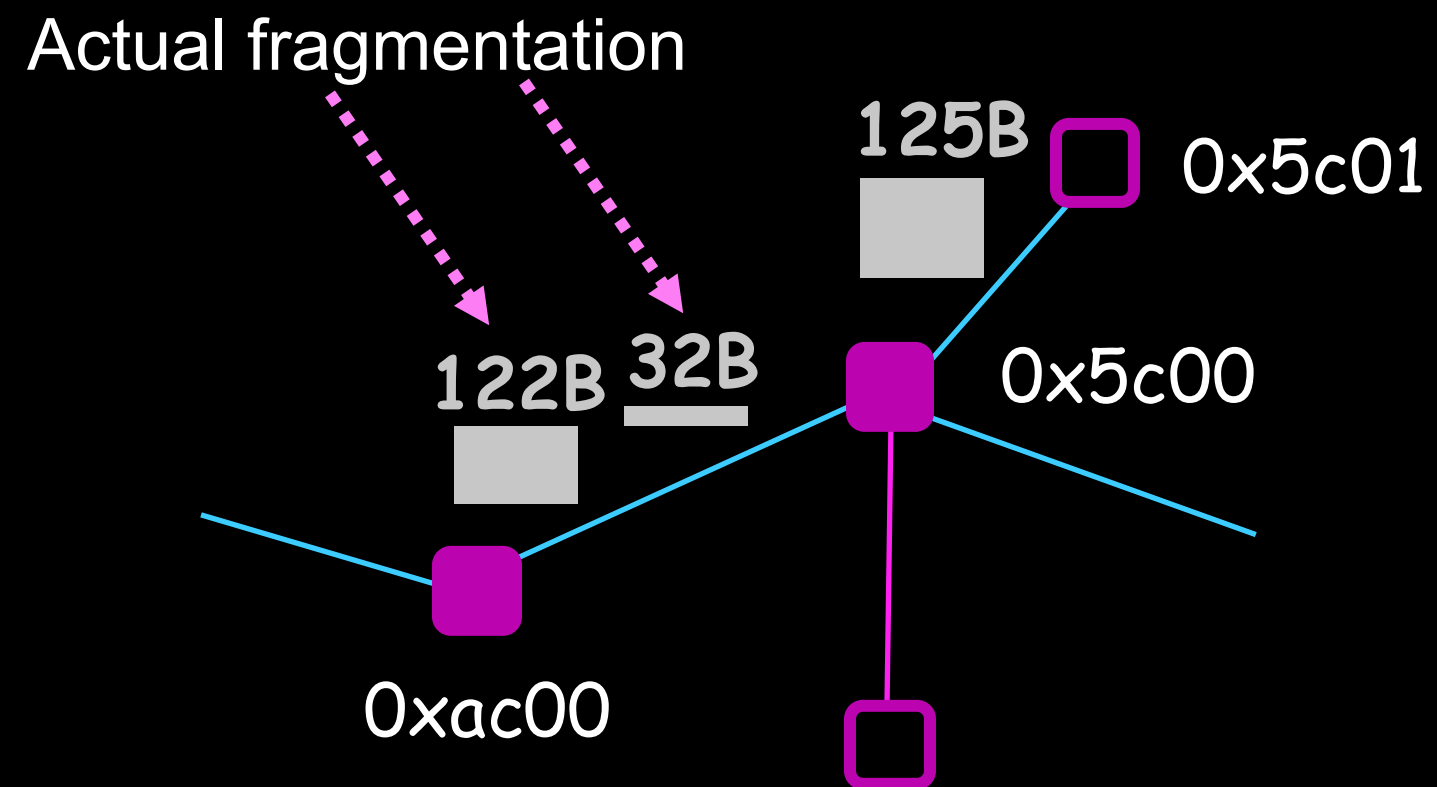
```
> IEEE 802.15.4 Data, Src: 0x5c00, Dst: 0xac00
└─ 6LoWPAN, Src: ::a2c5:f62:1ac0:80f0, Dest: fda0:13ed:8be5::be9
  └─ Fragmentation Header
    └─ 1100 0... = Pattern: First fragment (0x18)
    └─ Datagram size: 120
    └─ Datagram tag: 0xb824
  └─ IPHC Header
    └─ 011. .... = Pattern: IP header compression (0x03)
    └─ ...1 1... .... = Traffic class and flow label: Version, traffic
    └─ .... .1.. .... = Next header: Compressed
    └─ .... ..00 .... = Hop limit: Inline (0x0)
    └─ .... .... 1... .... = Context identifier extension: True
    └─ .... .... .1.. .... = Source address compression: Stateful
    └─ .... .... ..01 .... = Source address mode: 64-bits inline (0x0001)
    └─ .... .... .... 0... = Multicast address compression: False
    └─ .... .... .... .0.. = Destination address compression: Stateless
    └─ .... .... .... ..00 = Destination address mode: Inline (0x0000)
```

```
> IEEE 802.15.4 Data, Src: 0x5c00, Dst: 0xac00
└─ 6LoWPAN
  └─ Fragmentation Header
    └─ 1110 0... = Pattern: Fragment (0x1c)
    └─ Datagram size: 120
    └─ Datagram tag: 0xb824
    └─ Datagram offset: 112
  > [2 Message fragments (120 bytes): #271481(112), #271483(8)]
```



Fragmentation down the forwarding path

FRAGMENTATION DOES NOT ADD UP

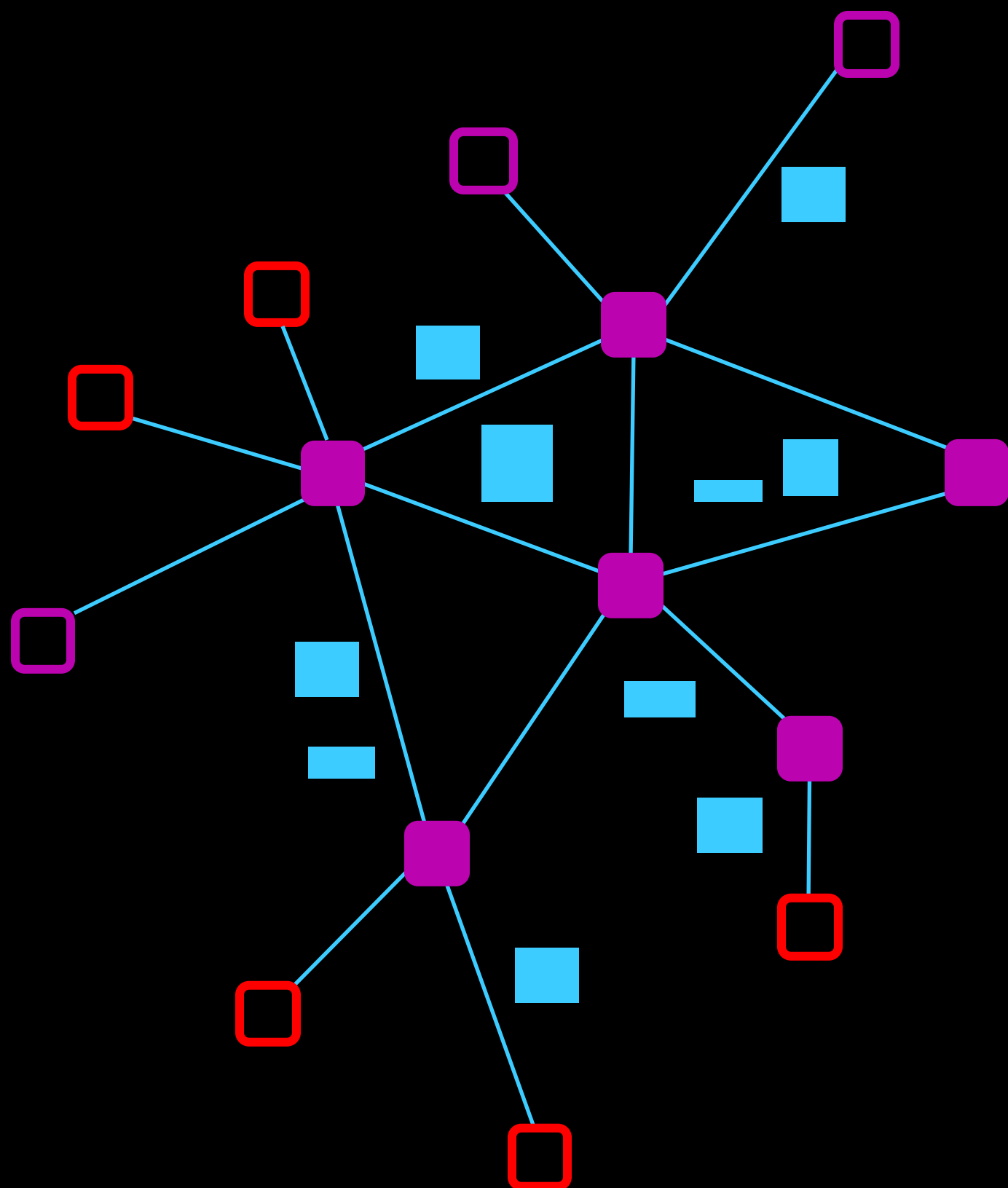


122 + 32 = 125 ?

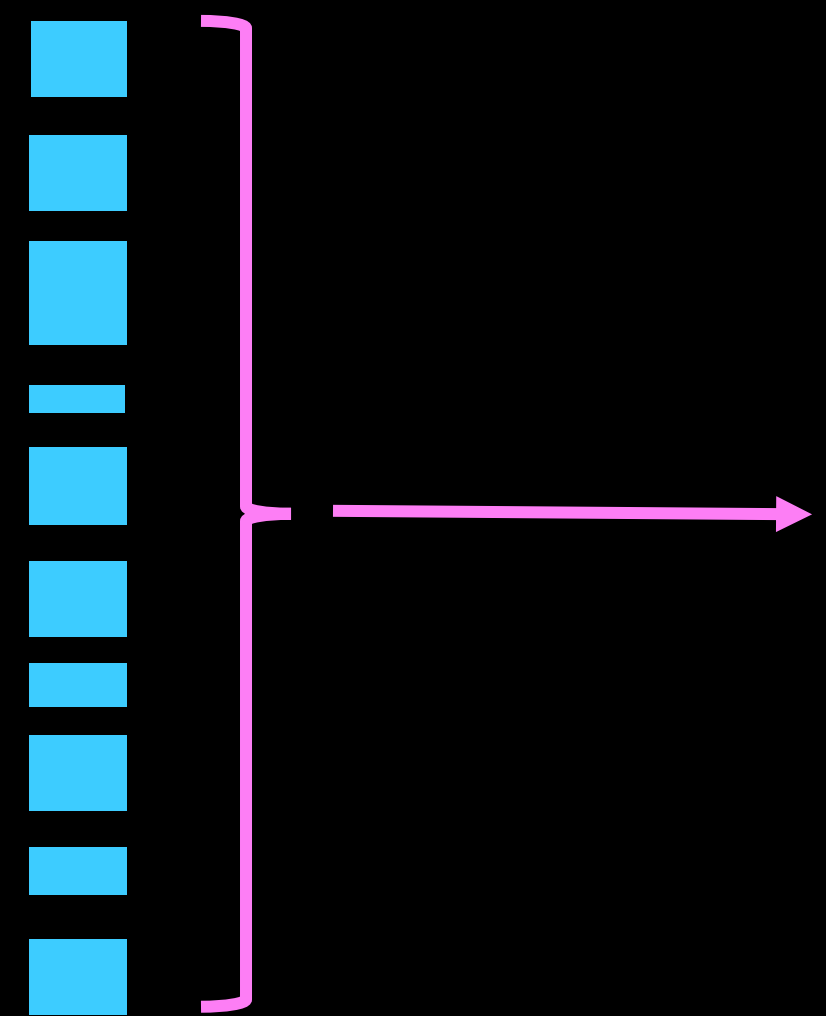




COMPLEX PROBLEM REQUIRES (COMPLEX) SOLUTION

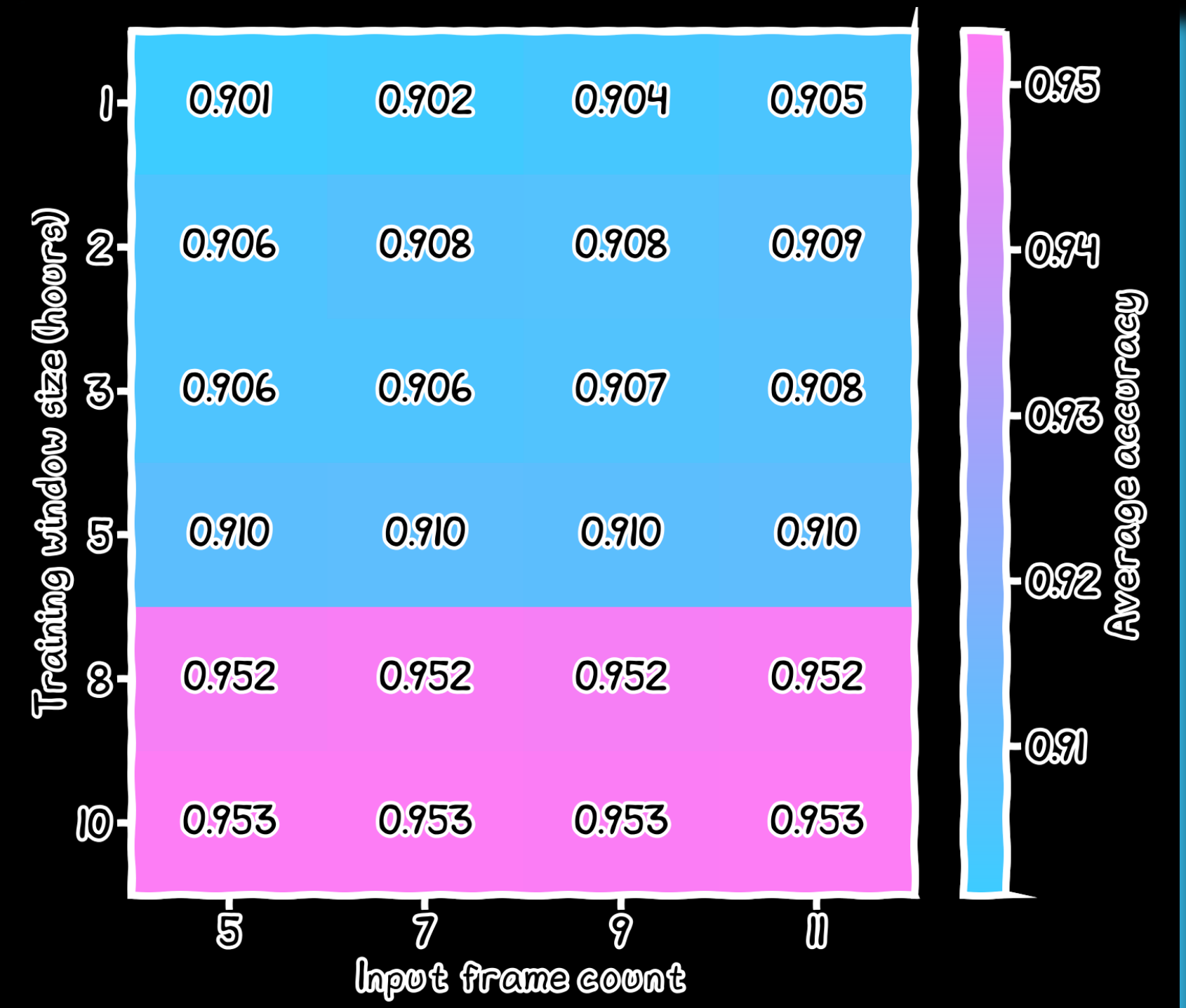
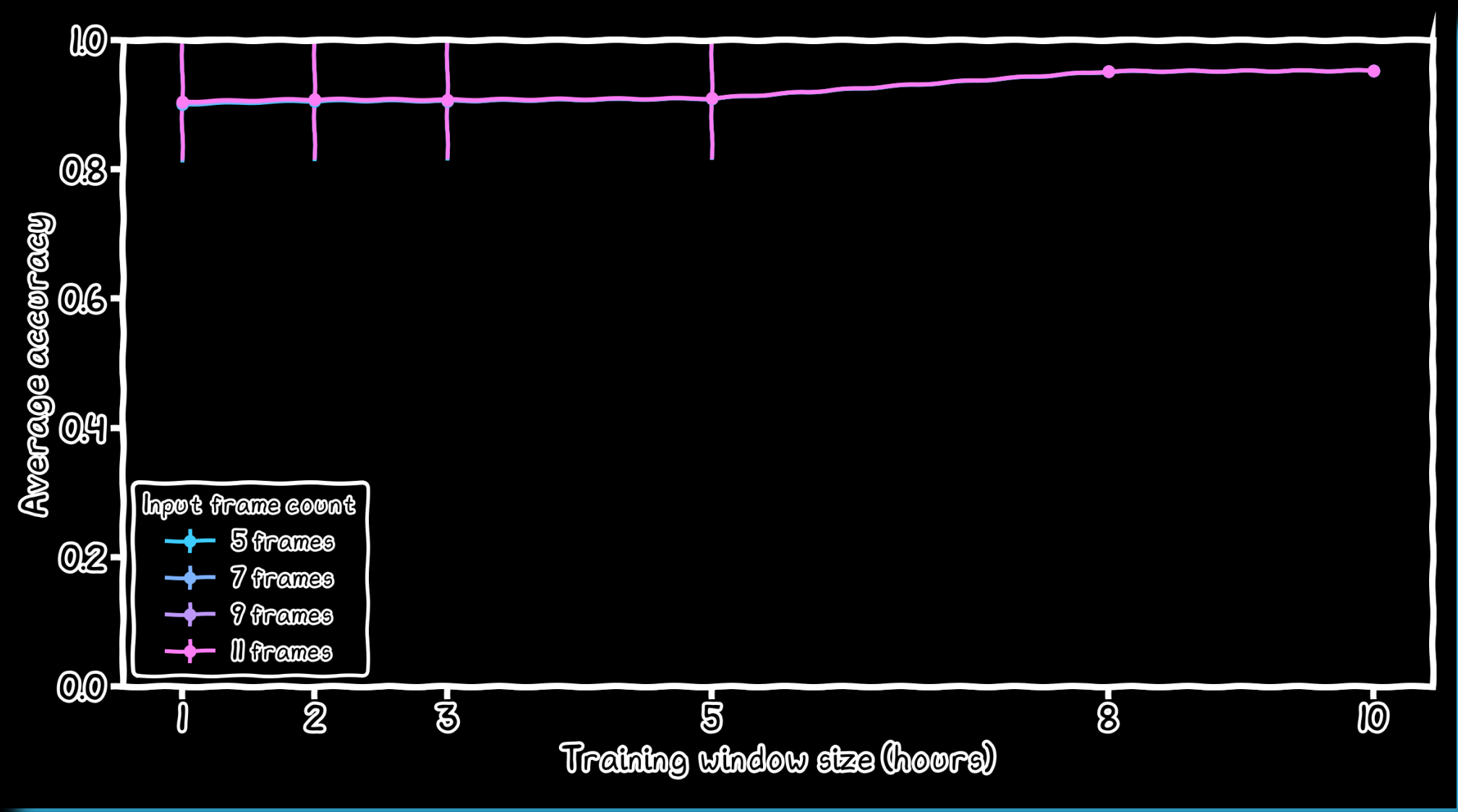


Machine learning to the rescue:
Random forests



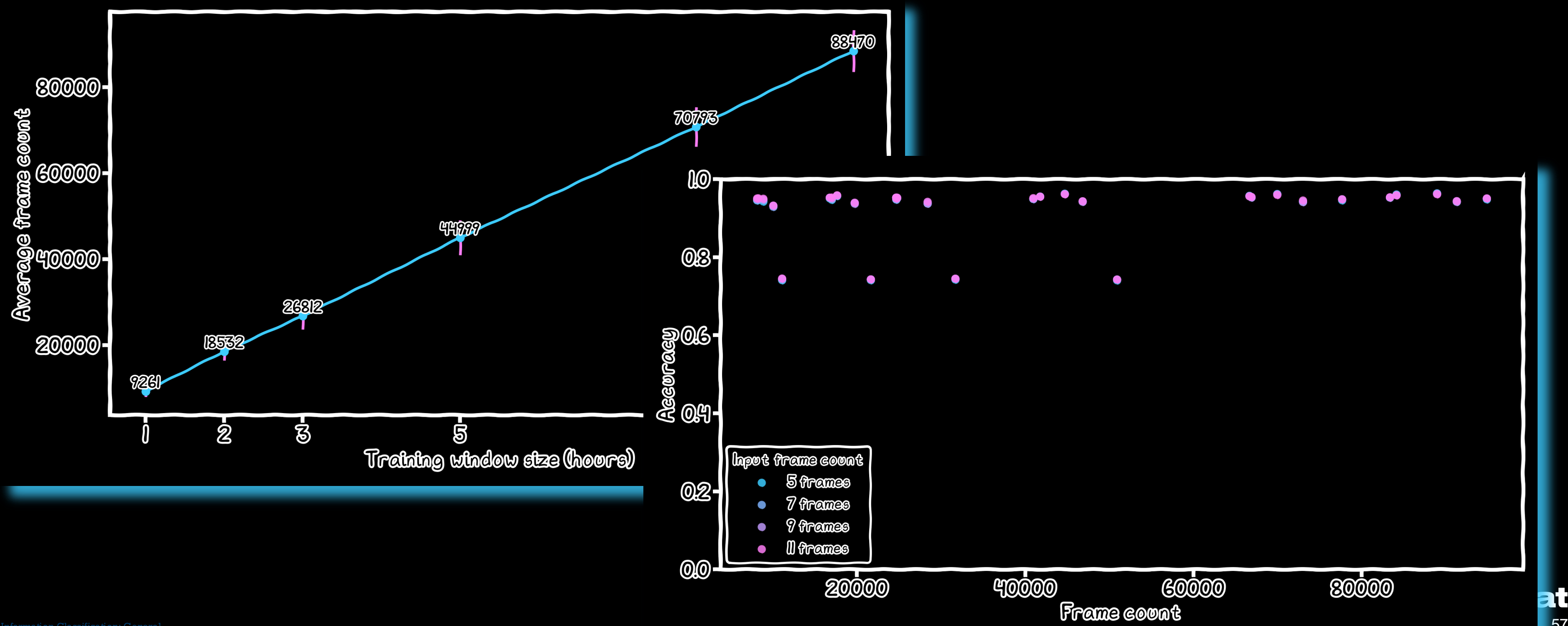
DEVICE CLASSIFICATION – MODEL TRAINING

- Much better, accuracy increases **above 95%**



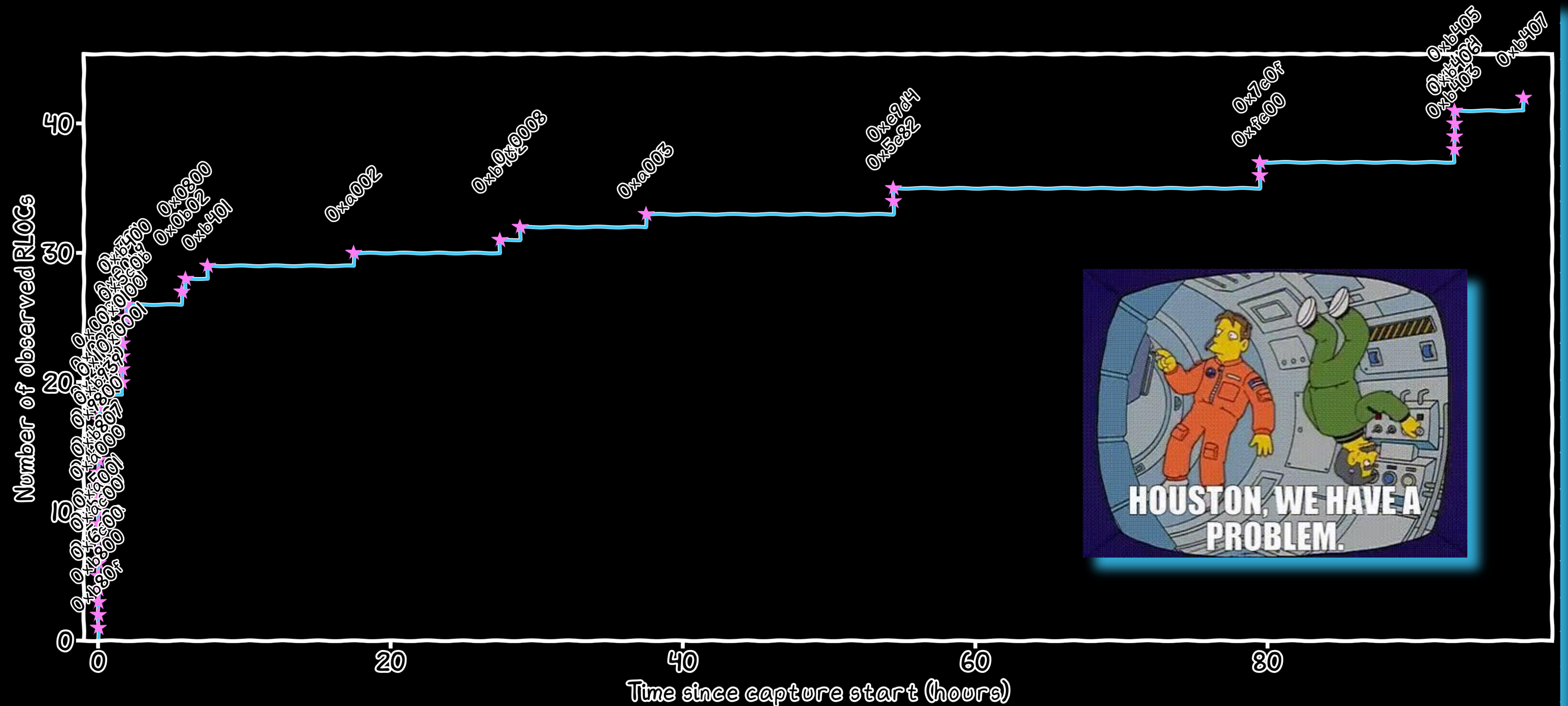
DEVICE CLASSIFICATION – HOW LONG TO WAIT

- The more time we sniff, the more frames we acquire
- It's more a question of having **the "right" frames**



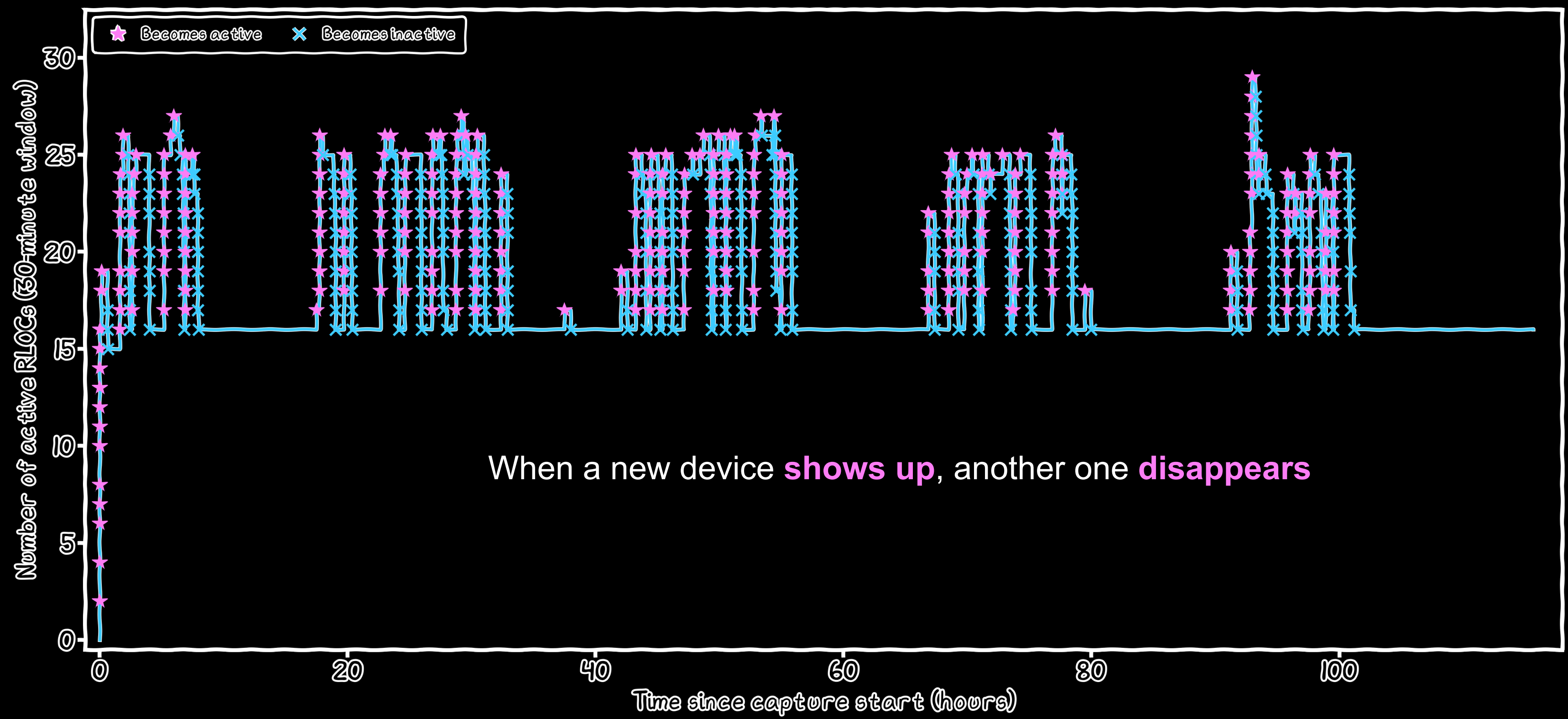
YAP: YET ANOTHER PROBLEM

We are seeing more devices than we know we have in the lab





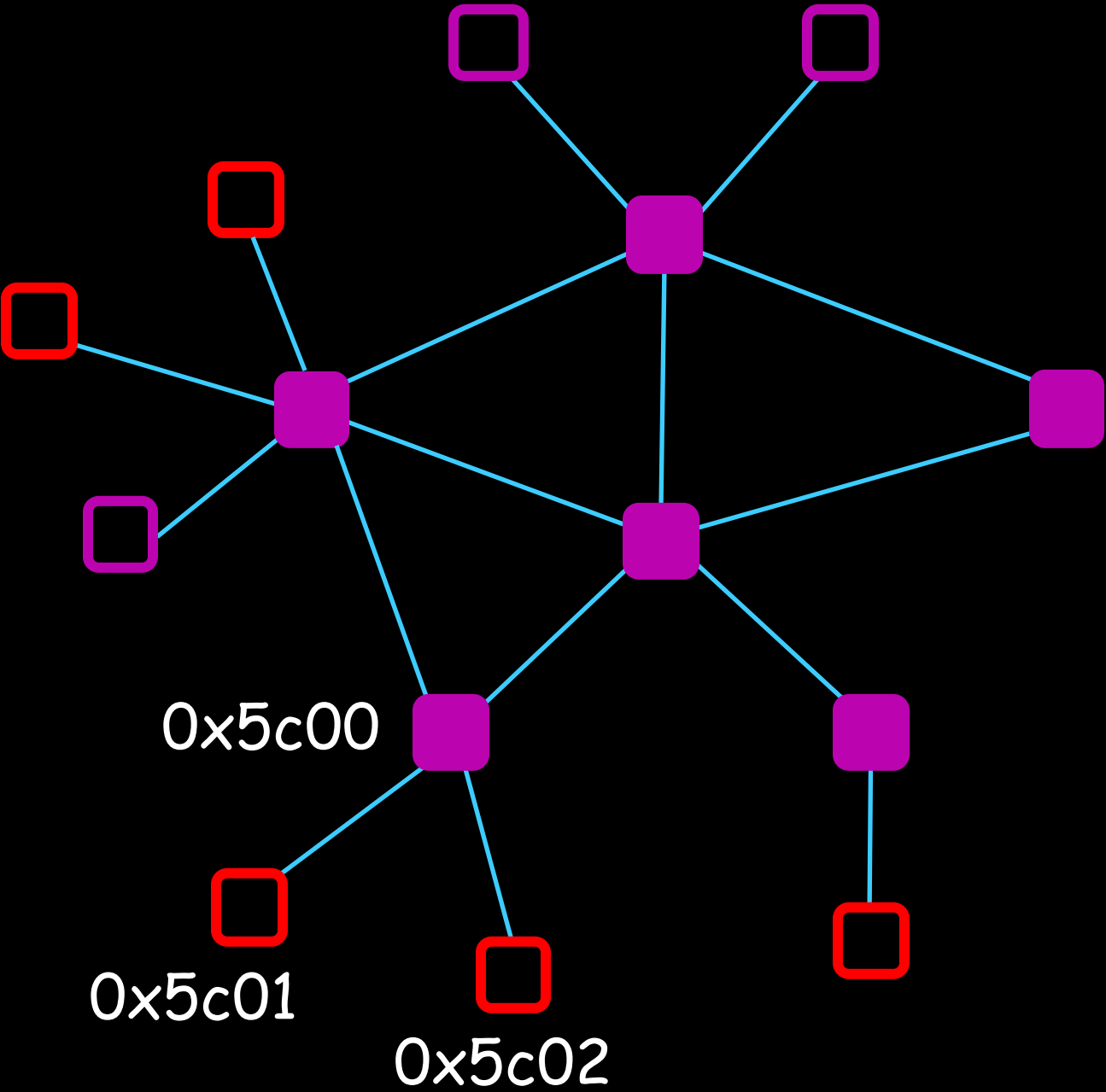
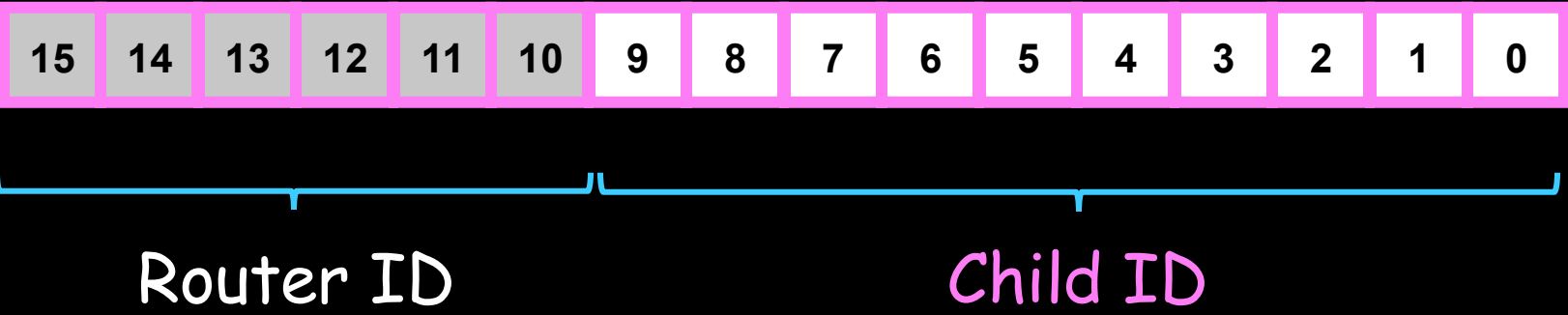
DEVICE COUNT IS CONSTANT



ROUTING LOCATOR 16: RLOC16

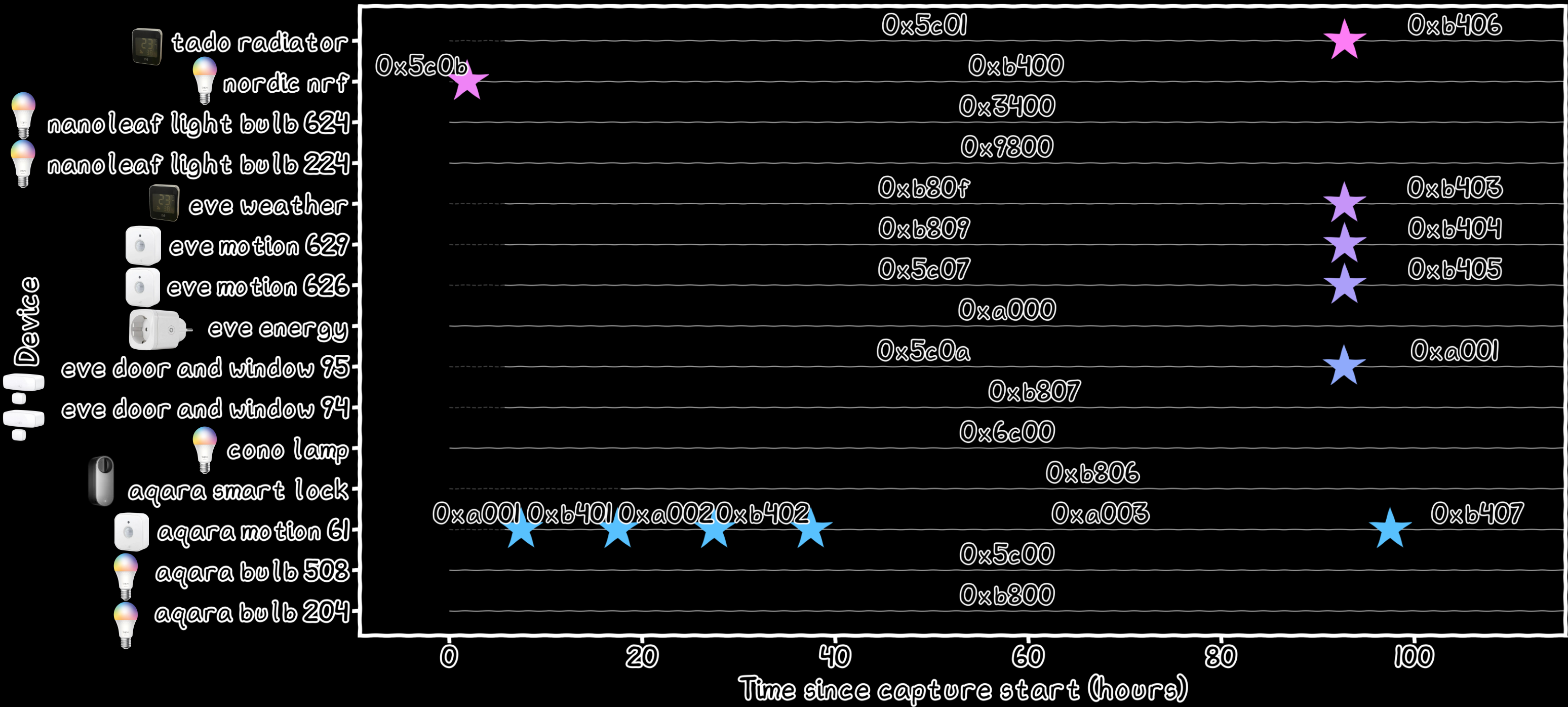
- 16-bit address assigned to every Thread device that identifies its location in the current Thread topology
- It is sent **non-encrypted**
- Router vs **Child**
- Router ID: upper 6 bits
- **Child ID**: lower 10 bits

$RLOC16 = (Router\ ID \ll 10) + Child\ ID$



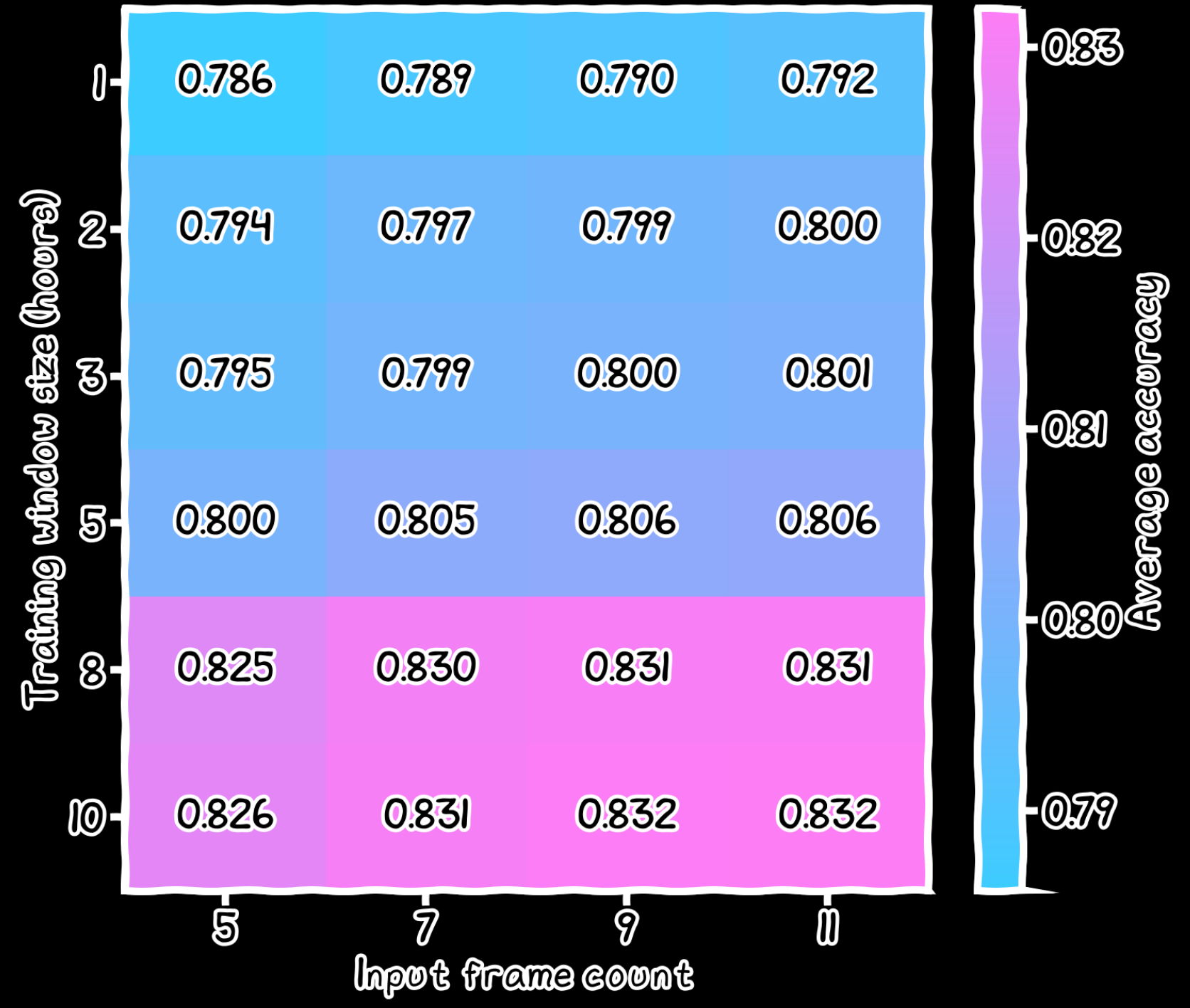
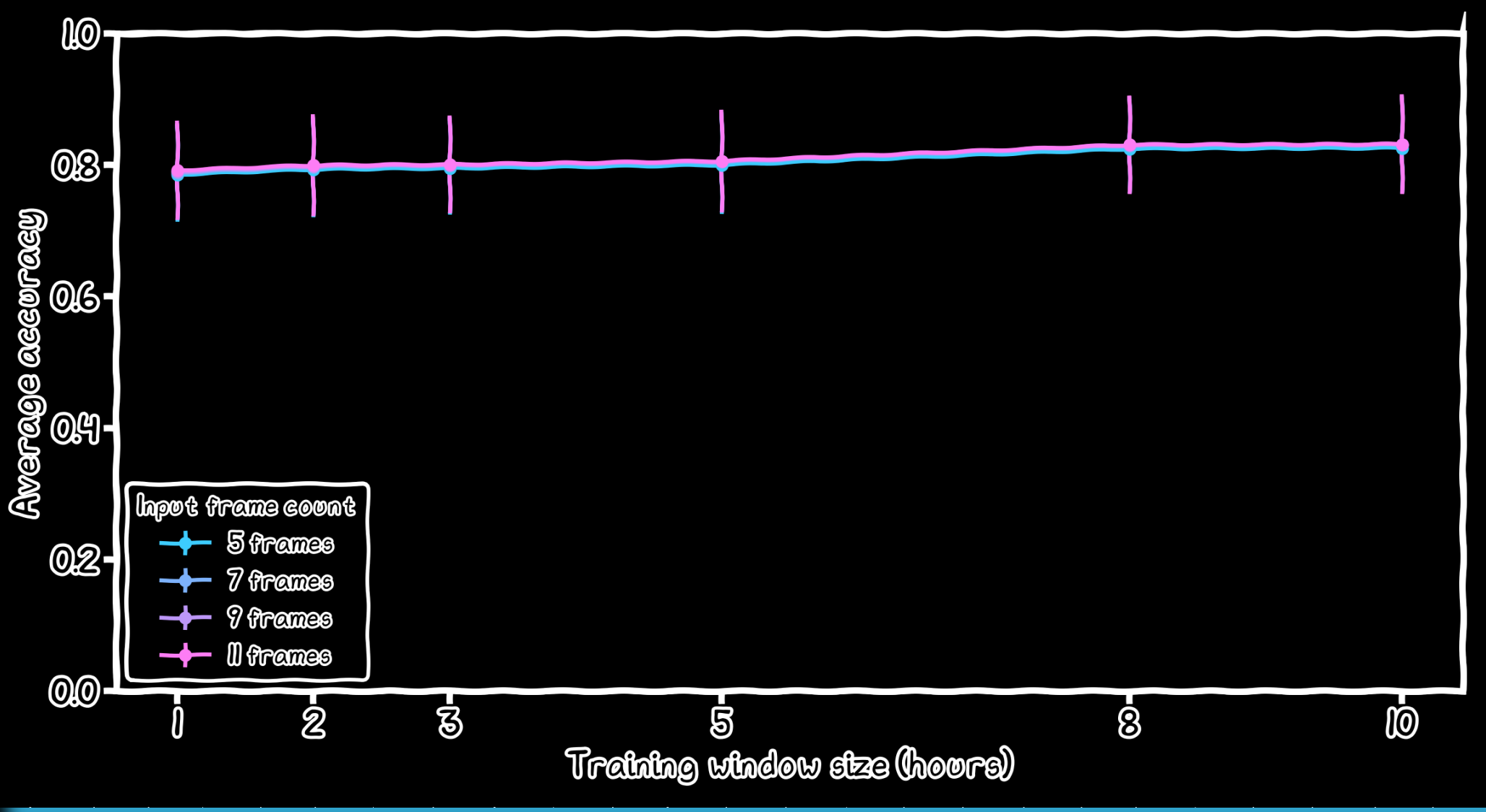
RLOC16 MAY CHANGE

- Depending on the device, network state, RLOC16 may change
- Some devices do not change their RLOC16 for weeks, others change it more frequently
- Solution: **fingerprinting**



DEVICE FINGERPRINTING

- Trained model
- The number of frames used as input does not have a major impact
- However, we seem to peak out in terms of accuracy at **~11 frames**



PRACTICAL EXPLOITATION



END-APPROACH

Thread
frame sniffing

Frame
forwarding
detection

Frame
variance
elimination

Frame
defragment

Device
classification

Device
fingerprinting



TARGET BUILDING: OFFICES

Thread network in a partner building from Târgu Mureș, Romania!



We **asked and got permission** to analyze one of the office's Thread setup!

Thread network



BUILDING SURVEILLANCE SCENARIOS



1

Inside the building, sniff other's offices

- Signal strength can be very good
- More frames, more details



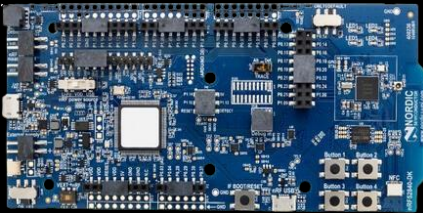
2

Outside the building, sniff whatever is leaking out

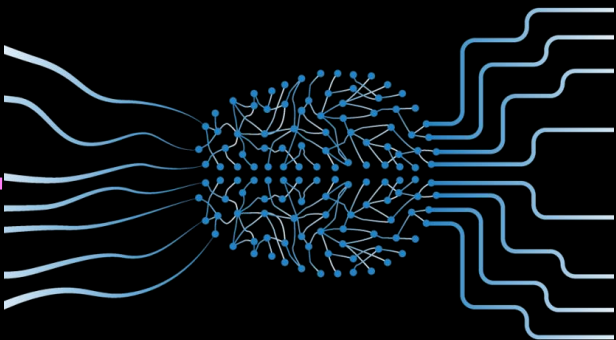
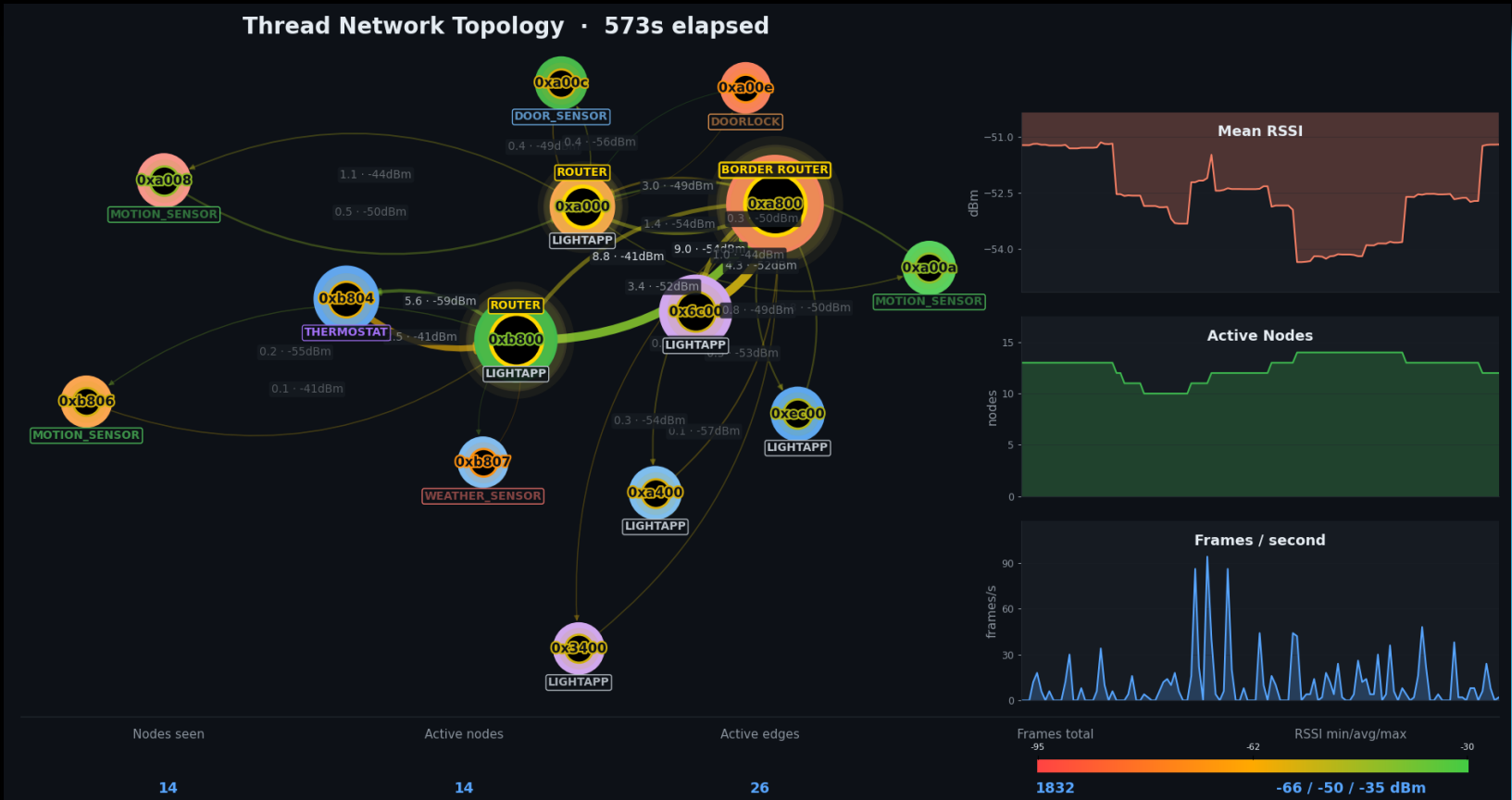
- Signal strength can vary, often significantly
- More time needed to capture (relevant) frames

INTEGRATED APPLICATION

Real-time Thread frame capture & analysis



Off-line Thread frame analysis



Integrates ML model for device identification



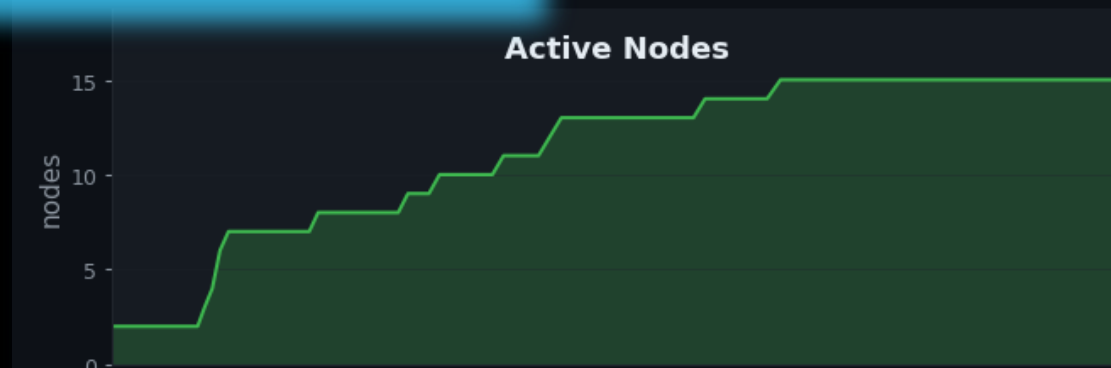
24 GHz 15M Band
Channels 11 to 25

Automated Thread channel
scan & detection

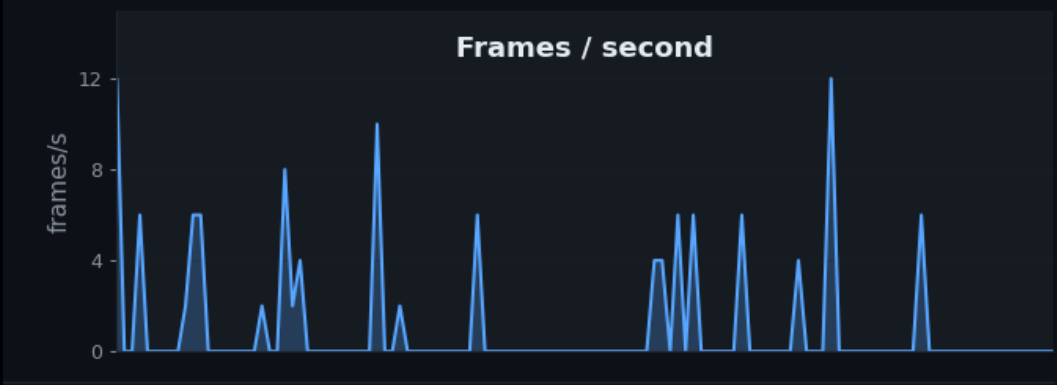
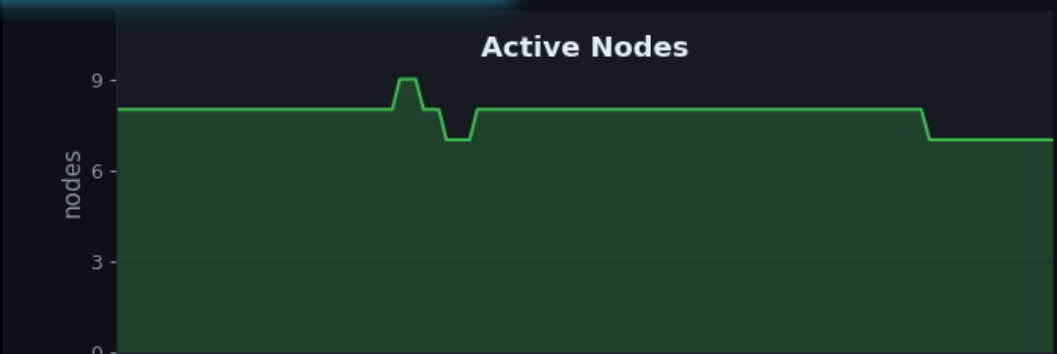
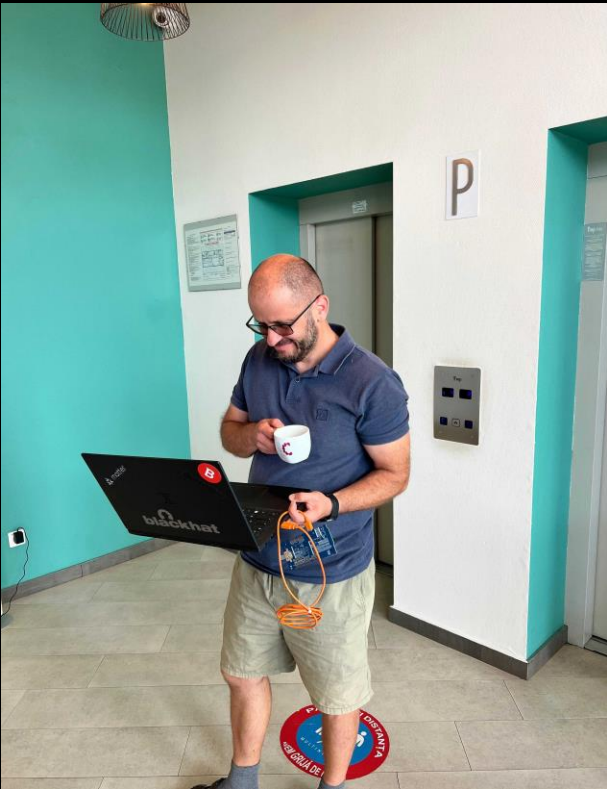
THREAD CHANNEL DISCOVERY

```
(venv) [bgenge@bgenge-1 BHUSA]$ sudo python dynamic_network_topology.py
Starting capture on /dev/ttyACM0 - auto-scanning channels for Thread traffic...
[capture] No channel specified - scanning for Thread traffic...
[scan]
[scan] 802.15.4 channel scan
[scan] Range : Ch 11-26 (16 channels)
[scan] Dwell : 2.0 s/channel
[scan] Est. : ~32 s total
[scan]
[scan] Ch 11 [ 1/16] 38 frame(s) ██████████ ◀ best so far
[scan] Ch 12 [ 2/16] 1 frame(s) █
[scan] Ch 13 [ 3/16] 0 frame(s) · (no traffic)
[scan] Ch 14 [ 4/16] 0 frame(s) · (no traffic)
[scan] Ch 15 [ 5/16] 1 frame(s) █
[scan] Ch 16 [ 6/16] 0 frame(s) · (no traffic)
[scan] Ch 17 [ 7/16] 0 frame(s) · (no traffic)
[scan] Ch 18 [ 8/16] 0 frame(s) · (no traffic)
[scan] Ch 19 [ 9/16] 0 frame(s) · (no traffic)
[scan] Ch 20 [10/16] 0 frame(s) · (no traffic)
[scan] Ch 21 [11/16] 0 frame(s) · (no traffic)
[scan] Ch 22 [12/16] 0 frame(s) · (no traffic)
[scan] Ch 23 [13/16] 0 frame(s) · (no traffic)
[scan] Ch 24 [14/16] 0 frame(s) · (no traffic)
[scan] Ch 25 [15/16] 2 frame(s) █
[scan] Ch 26 [16/16] 0 frame(s) · (no traffic)
```

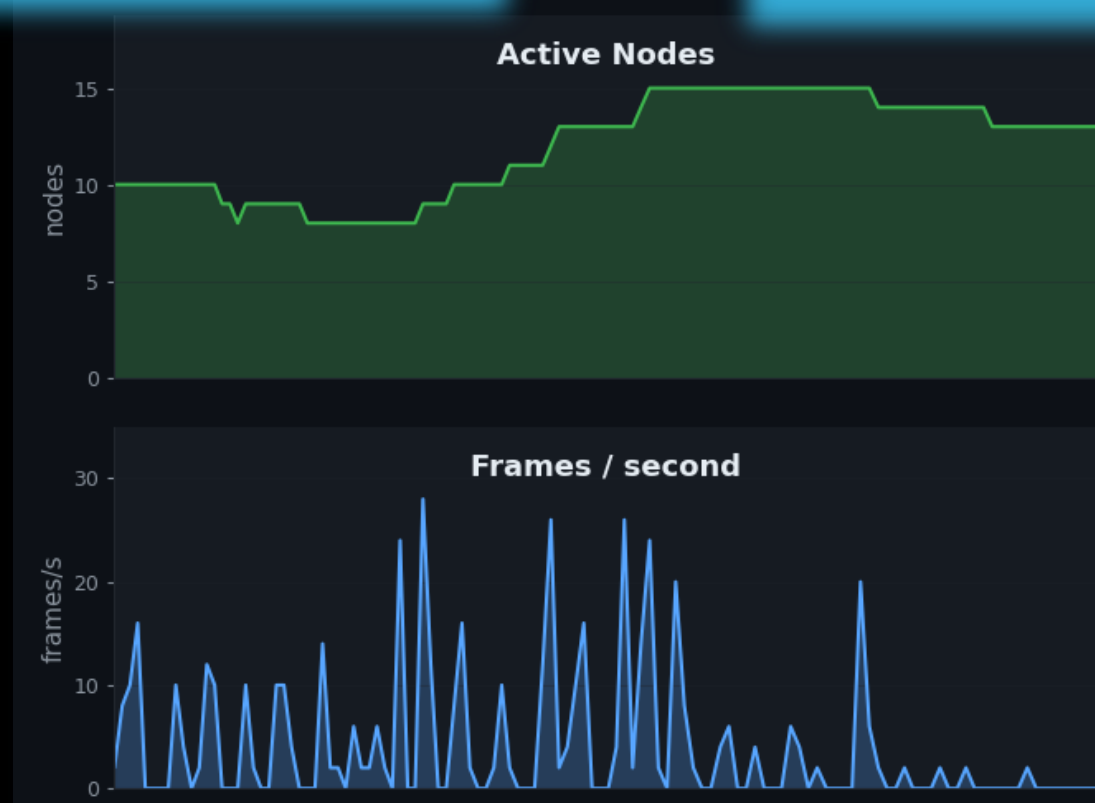
INSIDE THE BUILDING, OUTSIDE OF OFFICE



INSIDE THE BUILDING, FLOOR -2



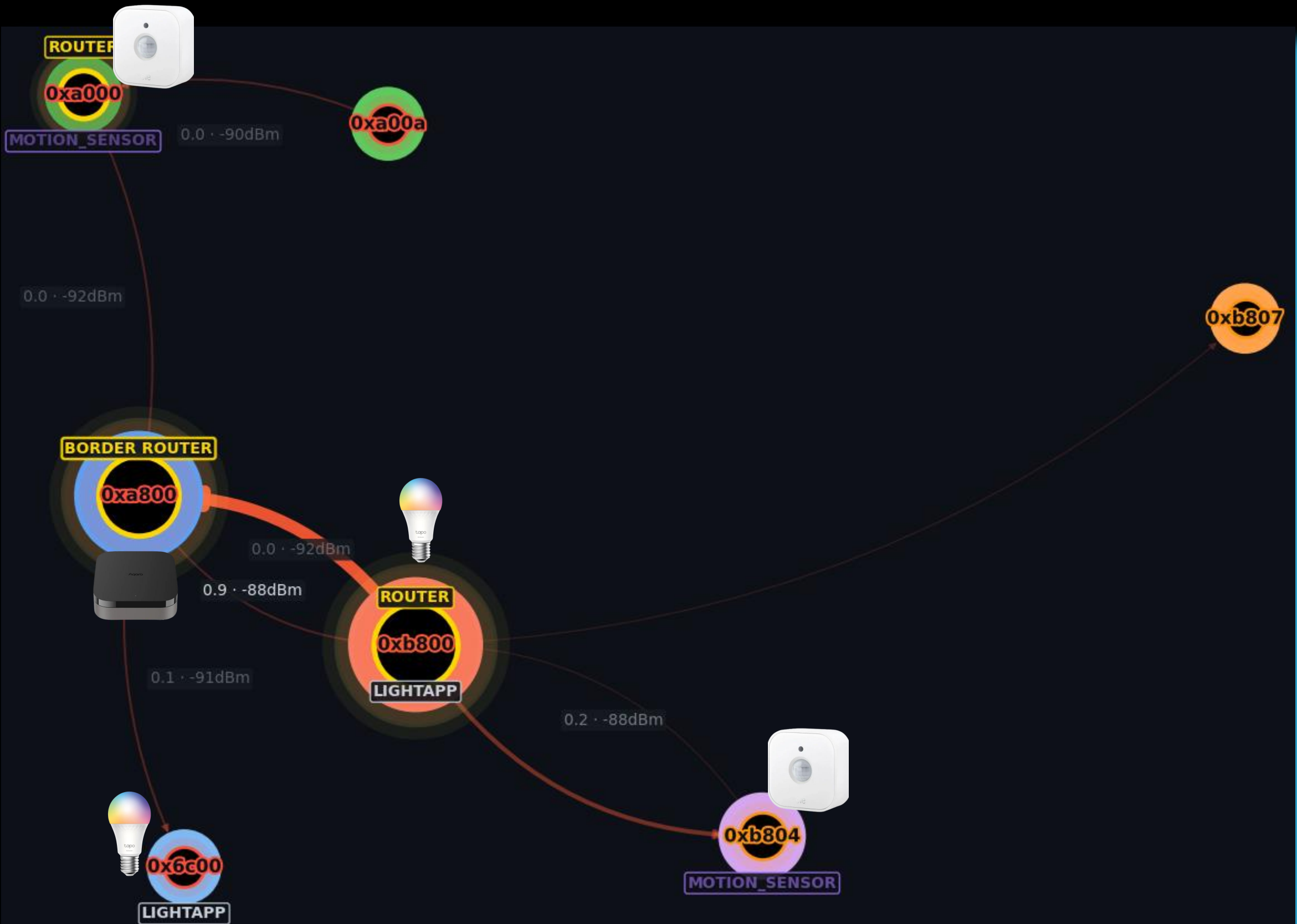
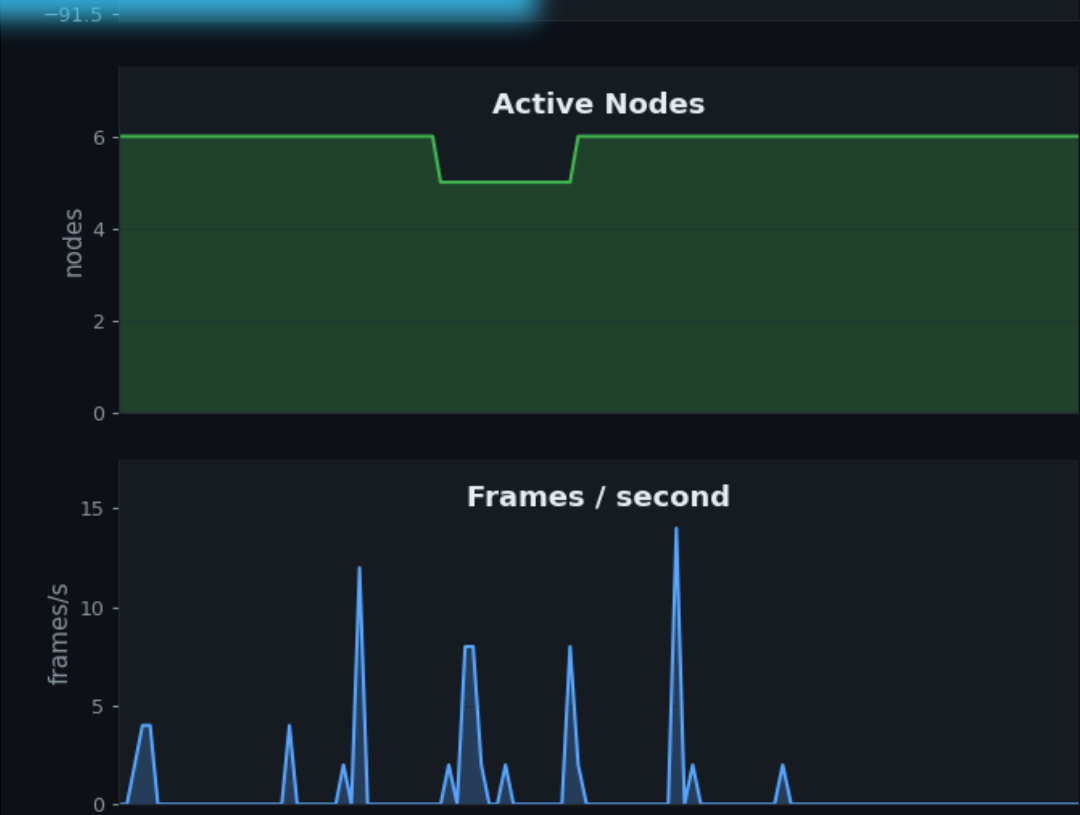
OUTSIDE THE BUILDING



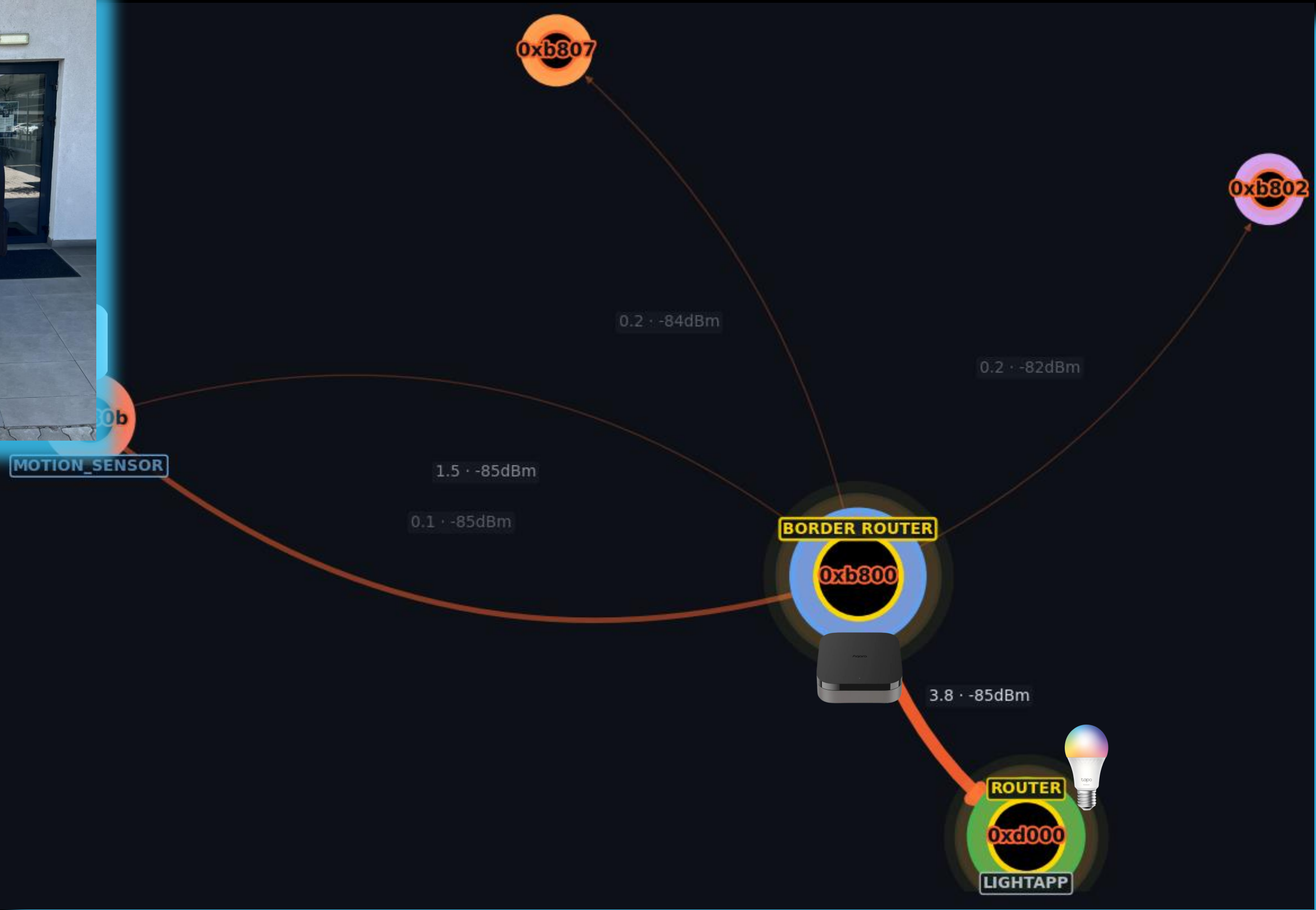
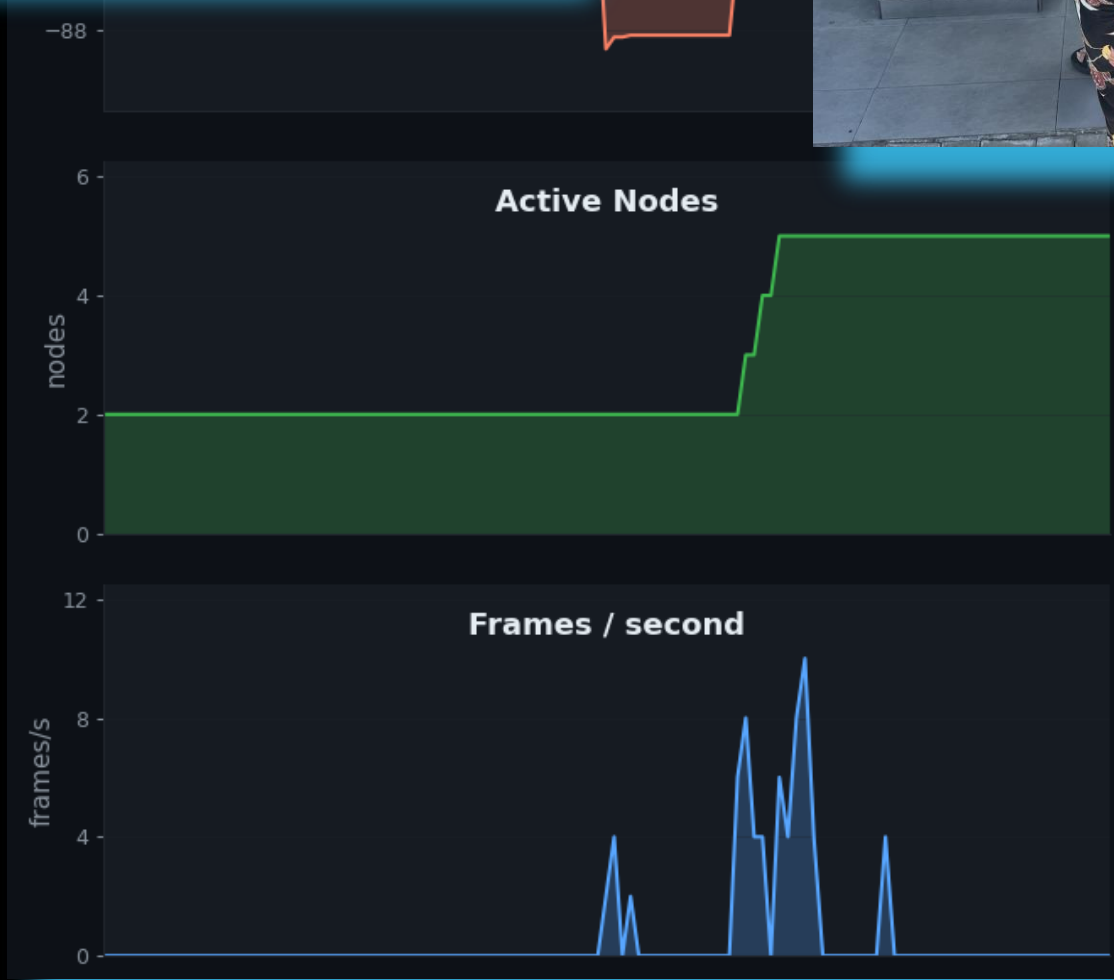
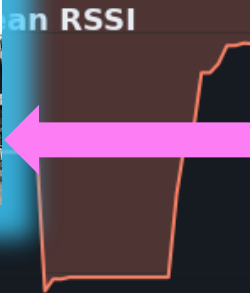
AT A NEARBY FAST FOOD ~30m DISTANCE



Target building



OTHER SIDE OF THE BUILDING



REMOTE MONITORING IS POSSIBLE

LOCATION, LOCATION, LOCATION

Thread network



The place to be

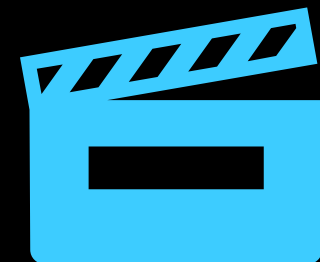
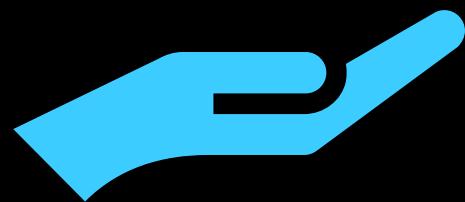


DEMO

THE INNOCENT BYSTANDERS

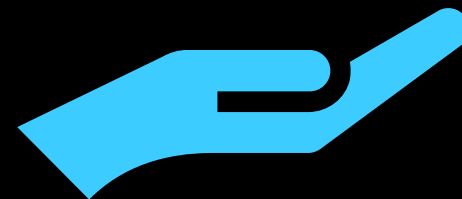


KEY TAKEAWAYS & ACTIONABLE ITEMS

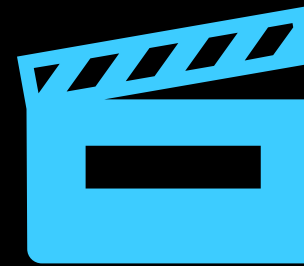


TAKEAWAYS

- 1 An increasing number of devices rely on wireless comms, making them inherently **susceptible to remote attacks**
- 2 **Insufficient protection** allows traffic capture, analysis, and information leakage. This can have serious privacy implications, leading to building and people profiling
- 3 **Protocols are typically not tested for "information leakage"** and therefore their design is not tailored to minimize information leakage



ACTIONABLE ITEMS



Protocol designers:

1 **Add security (cryptographic) protection** before moving towards critical applications (e.g., building automation / industry-grade applications)

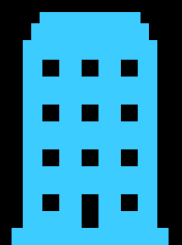


2 **Continue to research the security of Thread, Matter/Thread**; these are still new protocols and **likely have undiscovered attack surfaces**. They are also very popular (with steadily growing adoption), hence, vulnerabilities in these protocols may have significant or critical consequences

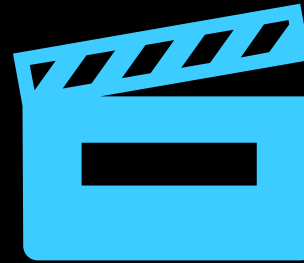


Building designers:

3 Carefully **consider the positioning of critical devices**, since communications can expose them to remote surveillance



ACTIONABLE ITEMS

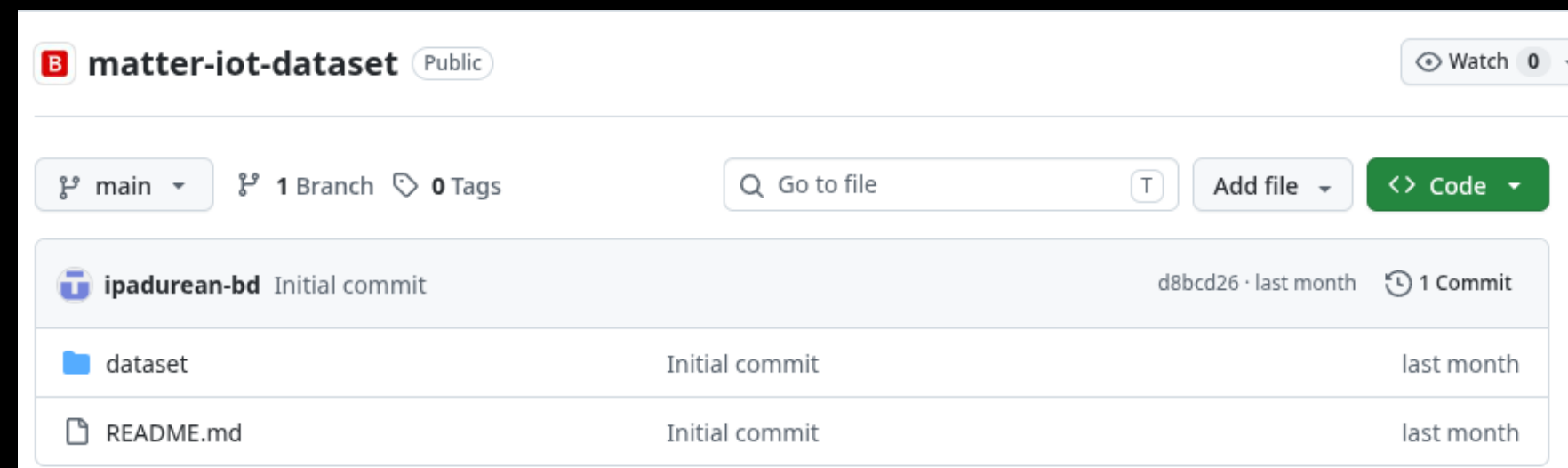


Everyone:

4

Take this research further! Application-layer metadata is now freely available to the security community!

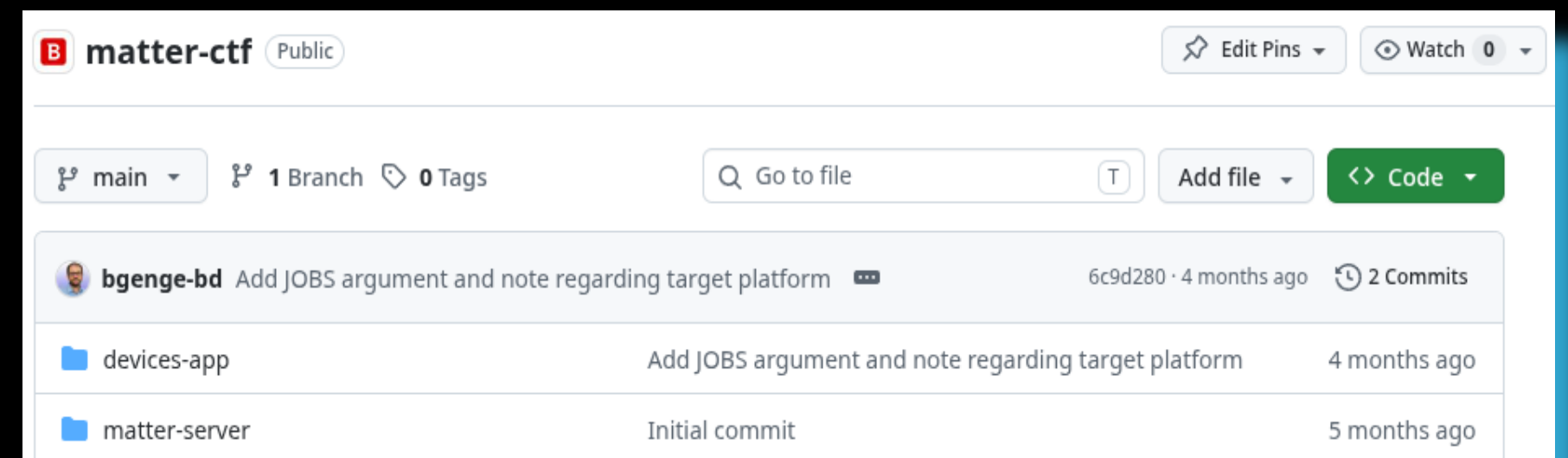
<https://github.com/bitdefender/matter-iot-dataset>



5

Create your own IoT virtual playground with realistic devices! We have published a Matter IoT framework!

<https://github.com/bitdefender/matter-ctf>



Thank you!

Questions?

bgenge@bitdefender.com, aburduv@bitdefender.com

Bitdefender®